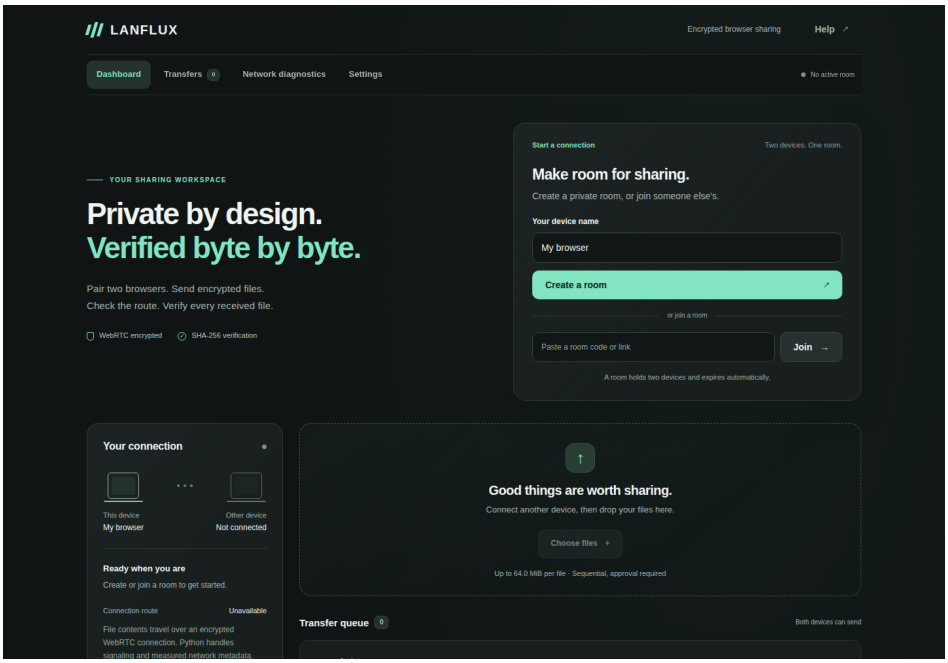
LANFLUX

AI-powered peer-to-peer file sharing

Technical report & operating guide

Browser networking, file integrity, security, machine learning, deployment and reproducible evaluation.



Documentation edition: 2026-10-22

Scope: Current implementation extending published version 0.3.0. Real browser evidence; no fabricated benchmark or accuracy claims.

Project: github.com/kasim672/LanFlux

Contents

01 / Project overview and report guide 3

02 / Installation and user guide 10

03 / Architecture and transfer protocol 16

04 / Machine learning and transfer intelligence 22

05 / Security and privacy 26

06 / Configuration reference 31

07 / HTTPS, TURN and public deployment 34

08 / Reproducible network experiments 39

09 / Verification evidence and limitations 44

10 / PDF font copyright and permission notice 47

Reading note

Start with the user guide to run the application. Architecture explains the networking protocol; the AI chapter separates measured observations, model outputs and advisory rules. Security, deployment and verification state the operational boundaries explicitly.

01 / Project overview and report guide

Maintained source: docs/REPORT.md

Project scope

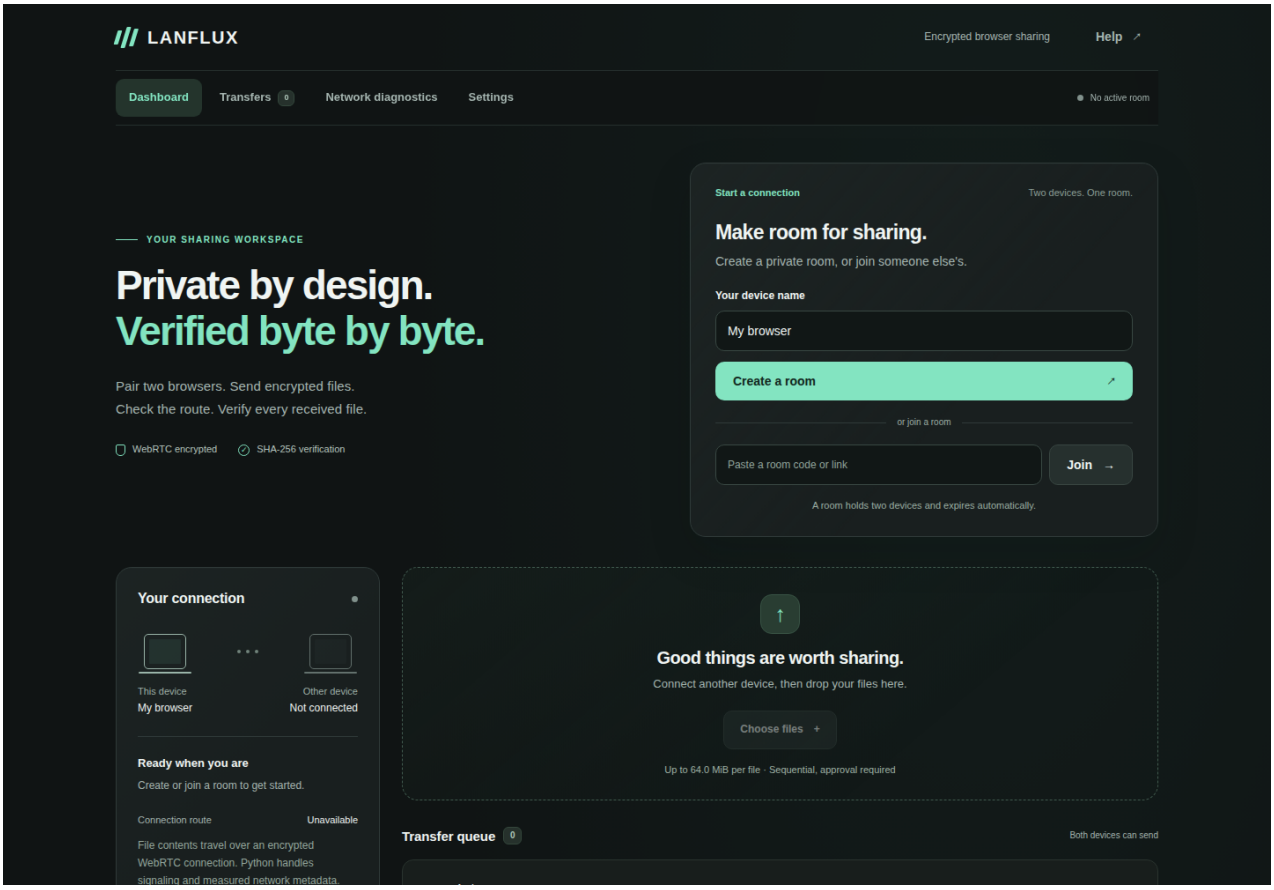
LANFLUX is a Python-backed, browser-to-browser file-sharing application with experimental network intelligence. Two devices pair through a temporary room, approve each file, transfer encrypted WebRTC payloads and verify SHA-256 before downloading. A separate, access-controlled metadata path learns network behavior using IsolationForest and RandomForestRegressor. The application remains useful when ML is collecting data or unavailable.

This report explains the implementation, reproducible operation, security model and observed verification. It is suitable for a Computer Networks and AI/ML project review. It contains no fabricated benchmark dataset, accuracy claim or screenshot. The current code extends published version 0.3.0; the report is not a claim that a new versioned release or public service has been provisioned.

Executive overview

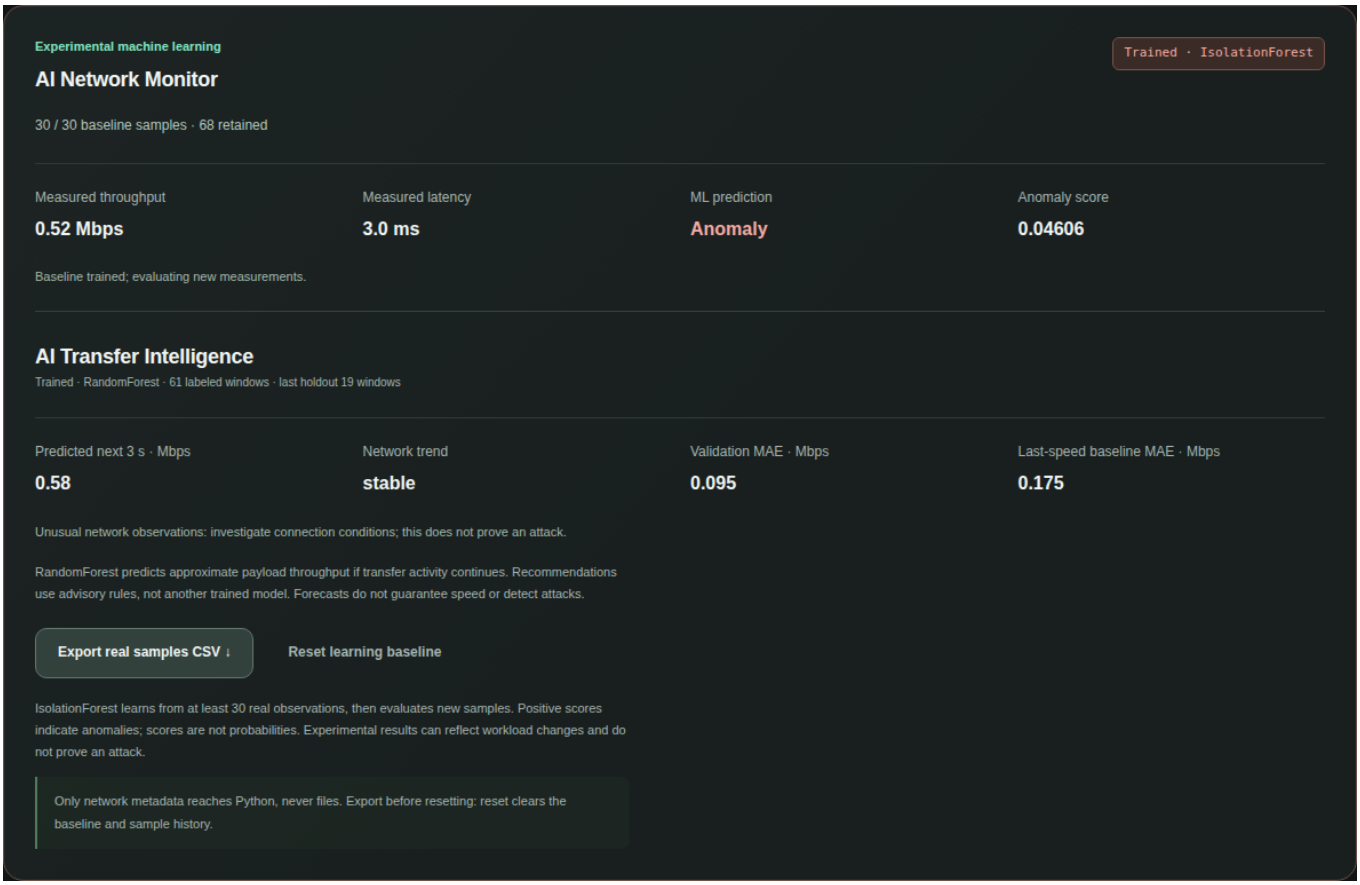
Concern	Implemented approach
Pairing	Unpredictable temporary two-peer rooms; matching QR/copy/display invitations; automatic query-link joining
Data transport	Reliable ordered WebRTC DataChannel with explicit approval, sequential queues, chunk limits and backpressure
Integrity	Incremental SHA-256 of original and received bytes; exact length validation before a download is offered
Discovery and fallback	Native ICE, optional STUN and secured TURN alternatives; actual selected-route diagnostics
Network visibility	Observed payload counters, monotonic timing, ping/pong RTT, bounded speed history and failure states
Unsupervised ML	Session-specific IsolationForest, real baseline collection, later signed anomaly scores
Supervised ML	Past-only RandomForest three-second throughput forecast, actual future labels and purged chronological validation
Interpretation	Explicit advisory rules combining fresh forecasts and anomaly observations; no automatic transfer control
Deployment	Local loopback default; explicit LAN mode; documented HTTPS/WSS reverse proxy and external coturn configuration

Real interface evidence



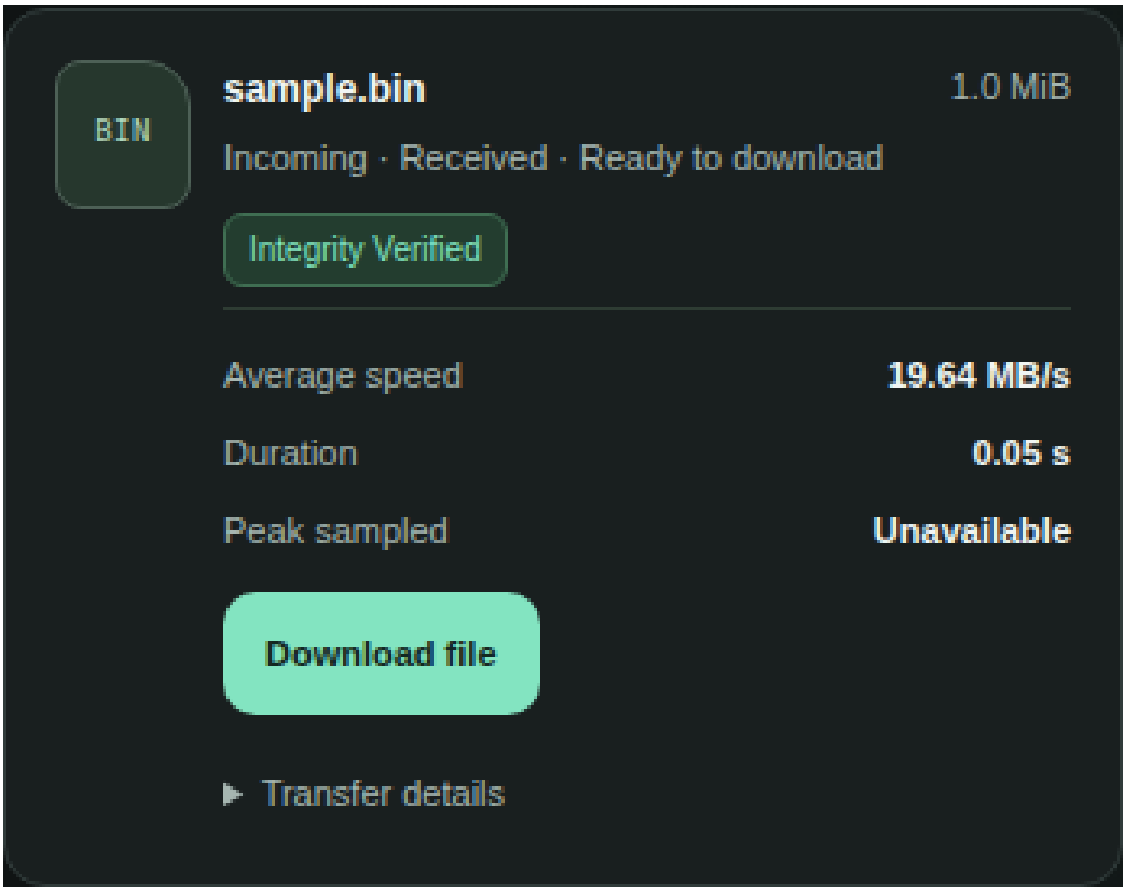
Actual LANFLUX homepage captured from the running application

The homepage and sharing screenshots were refreshed on 22 October 2026 in real Chromium application scenarios. The complete capture manifest is in the screenshot gallery. Room capabilities and invitation addresses are masked where present. Narrow-screen images demonstrate viewport responsiveness, not a physical-device certification.

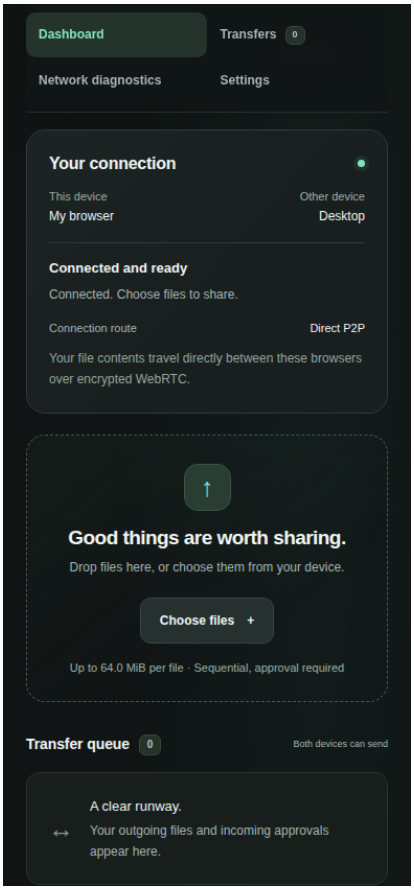


Actual AI panel with measured throughput, RTT and trained models

The AI panel shows an actual controlled loopback WebRTC transfer with test-only sender pacing. Displayed values were produced by observations and fitted models. They demonstrate the implemented pipeline; they are not representative Wi-Fi performance, labeled anomaly accuracy or proof that RandomForest improves network speed. Test-only pacing is absent from normal application operation.



Actual successful received file with verified SHA-256



Actual narrow mobile viewport of the working interface

The successful transfer image reflects real received and verified bytes. The mobile view is emulated Chromium; it does not establish current physical-phone or Safari compatibility.

How to read this report

The subsequent chapters reproduce the maintained repository guides: user operation; system architecture and transfer protocol; AI algorithms and evaluation; security; configuration; public deployment; network experiments; and executed verification. The Markdown guides are the canonical source. Changes to those guides should be followed by a PDF rebuild.

Engineering conclusions and future work

The working system separates transport, signaling and metadata analysis. File sharing depends on browser connection and receiver consent, while optional ML can remain collecting or unavailable without blocking it. Integrity checks and observed route labels make successful transfers auditable without pretending to verify the sender's real identity.

The current implementation deliberately keeps a bounded in-memory MVP. Future improvements require their own implementation and evidence:

- Streaming-to-disk on browsers with a suitable file API, with a safe fallback elsewhere.

- Safely resumable transfers with explicit protocol/version and retained-state design.

- More physical mobile devices and Safari/Firefox regression coverage.

- Longer, independently collected network sessions and labeled anomaly evaluation.

- A truly separate out-of-session test set for throughput forecasting and comparison against stronger baselines.

- Optional pacing experiments only after measured evidence and safe transfer-engine boundaries; Smart Mode is absent today.

- Shared room/session infrastructure before any multi-worker or multi-instance scaling.

- Operational relay-secret rotation, public access controls and approved campus deployment validation.

These are proposed extensions, not current capabilities or promised performance improvements.

Terms and units

Term	Meaning in LANFLUX
Signaling	Room admission and SDP/ICE exchange used to establish the browser connection
ICE	Browser connectivity checks selecting a permitted candidate pair
STUN	Discovery of mapped connection addresses; does not relay file payload
TURN	External relay for encrypted WebRTC traffic when a suitable direct route is unavailable
DataChannel	Ordered, reliable browser-to-browser message transport used for control and file chunks
RTT	DataChannel ping/pong elapsed time, including browser scheduling and channel queueing
MiB	Binary file-size unit: 1,048,576 bytes
MB/s	Decimal payload rate: 1,000,000 bytes per second
Mbps	Decimal payload rate: 1,000,000 bits per second; eight times decimal MB/s
SHA-256	256-bit digest used to compare original and received file bytes
IsolationForest	Unsupervised model identifying observations unusual relative to its collected baseline
RandomForestRegressor	Supervised ensemble predicting an approximate future measured payload rate
MAE	Mean absolute prediction error in Mbps; lower is better on the stated validation observations
Baseline	A defined comparison: collected observations for IsolationForest, or last measured speed for forecast error
Capability	A secret possession-based room identifier; it does not establish a person's real-world identity

Rebuild the report

The final PDF is committed at `docs/LANFLUX_Technical_Report.pdf`. The generator reads these Markdown guides and embeds the real screenshots; no external AI service or document upload is used.

```
python -m pip install -r requirements-report.txt
python scripts/build_report.py
```

The builder needs Python 3.10+ and ReportLab. It uses bundled ReportLab fonts for portability, with no application dependency changes. The result includes a contents page, chapter bookmarks, links and numbered pages. For visual verification, render it with Poppler:

```
pdftoppm -r 80 -png docs/LANFLUX_Technical_Report.pdf output/report-review/page
```

Create `output/report-review` first. Generated previews are ignored and should not be committed. The PDF is documentation, not a model artifact; models, observations, secrets and received files remain excluded from the repository.

Technical references

The implementation source and tests are primary evidence for LANFLUX-specific behavior. The following official references explain its underlying technologies; they are not benchmark or compatibility certifications.

- W3C WebRTC API specification: peer connections, ICE configuration, SCTP capabilities, statistics and DataChannel buffer APIs.

- IETF RFC 8831: WebRTC Data Channels: non-media peer data transport using SCTP in WebRTC.

- scikit-learn IsolationForest: unsupervised isolation model, decision function and constructor parameters.

- scikit-learn RandomForestRegressor: supervised tree ensemble and bounded fitting parameters.

- FastAPI documentation: Python HTTP/WebSocket application framework.

- coturn REST authentication: timestamped usernames and HMAC relay credentials.

- Nginx WebSocket proxying: explicit protocol-upgrade forwarding.

Runtime dependencies are declared in `requirements.txt`; tests in `requirements-dev.txt` and `package.json`; optional report generation in `requirements-report.txt`. Library versions may change within declared ranges. Reproduce an evaluation with recorded installed versions and the exact Git commit rather than assuming every dependency resolution is identical.

02 / Installation and user guide

Maintained source: docs/USER_GUIDE.md

LANFLUX shares files between two ordinary browsers using an encrypted WebRTC DataChannel. Python provides the website, temporary rooms, signaling and experimental network analysis. File contents, filenames and file SHA-256 digests do not pass through FastAPI. A direct route connects the browsers; a TURN route forwards their encrypted WebRTC traffic through a configured relay.

This guide describes implemented behavior. See configuration for every setting, deployment for public hosting, and AI monitor for model details.

1. Install and start

Use Python 3.10 or newer. Install dependencies on the computer running the server; the receiving phone needs only a browser. Node.js is required for development tests, not normal use.

```
git clone https://github.com/kasim672/LanFlux.git
cd LanFlux
```

Linux and macOS

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m lanflux
```

If an existing checkout already contains a virtual environment, activate it and install the current requirements rather than cloning another copy. Some Linux distributions require their `python3-venv` package before creating an environment.

Windows PowerShell

These commands do not require changing the PowerShell activation policy:

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m lanflux
```

Open **http://localhost:8000** on the server computer. The default bind address is `127.0.0.1`, so this mode is intentionally reachable only on that computer. **http://localhost:8000/health** is the health endpoint. Stop the server with Ctrl+C; restarting clears temporary rooms and AI sessions.

Enable trusted local Wi-Fi access

```
python -m lanflux --lan --port 8000
```

On Windows:

```
.\.venv\Scripts\python.exe -m lanflux --lan --port 8000
```

`--lan` binds all network interfaces. Use a trusted network and allow Python/TCP port 8000 through the computer's firewall for the appropriate private network. Do not expose this development port publicly. Changing `--port` also changes the port used in invitations. Keep one Python worker: room membership and

models live in one process.

Optional configuration

Copy `.env.example` to `.env`, edit only the needed settings, then restart. The CLI loads `.env` automatically; `--env-file PATH` selects another file. Keep secrets and private keys outside Git.

```
python -m lanflux --help
```

For public use, provide HTTPS/WSS hosting and a trusted public invitation origin. A website being reachable does not guarantee WebRTC connectivity; separate TURN infrastructure may be needed. Follow the HTTPS/WSS and coturn deployment guide.

2. Pair the computer and phone

1. Put both devices on the same trusted Wi-Fi. Keep both browsers foregrounded and prevent the phone from sleeping during the test.

2. Start LANFLUX with `--lan`. Open `http://localhost:8000` on the computer.

3. Select **Create a room**. LANFLUX generates an unpredictable temporary room code.

4. Open **QR code** and scan it with the phone camera. Alternatively use **Copy invite link**, or open the displayed LAN URL on the phone and enter the room code.

5. Opening an invitation containing `?room=CODE` automatically attempts to join that room. No second manual code entry is required.

6. Wait for **Connected and ready** before choosing files. Both devices can now send.

The QR image, copied invitation and displayed URL refer to the same room. When the computer opened localhost, LANFLUX attempts to discover a private IPv4 address on a physical network adapter, excluding Docker, VPN and virtual interfaces. It preserves the configured port. A phone's `localhost` means the phone itself, so an invitation containing localhost will not reach the computer.

If adapter discovery is unavailable or selects an unreachable physical network, open LANFLUX through the computer's reachable LAN address before creating a room, or set this in `.env`:

```
LANFLUX_PUBLIC_URL=http://YOUR_REACHABLE_LAN_IP:8000
```

Replace the placeholder with the actual address; do not copy it literally. Restart and create a **new room**. Existing invitations remain pinned for consistency. On a public deployment this setting should be the actual HTTPS origin, without a path.

Treat room links and QR images as private invitations. A room admits two peers and expires after one hour by default. Invalid, expired or full rooms produce an error; ask the other person to leave or create a fresh room. Do not repeatedly retry an invalid code: admission requests are rate-limited.

3. Send, approve and download

1. On either device, select **Choose files**. Desktop users can also drag files into the drop area.

2. LANFLUX reads bounded slices to calculate the original SHA-256 digest. Hashing can take time on a slower phone.

3. The receiving device sees an incoming offer with the file's name and size. Select **Accept** or **Reject**. File payload does not start before approval.

4. Watch transfer progress, status and observed rate. Multiple outgoing files run sequentially; the opposite device has its own independent queue.

5. The receiver checks the exact byte count and computes SHA-256 over the received bytes.

6. Integrity Verified appears only after both checks succeed. Use the download control to save the completed file.

A digest match proves that the received bytes match the peer's announced digest. It does not prove the sender's real-world identity, certify a file as safe, or replace your device's normal precautions for opening files.

Cancel, reject, leave and restart

- Reject declines an incoming offer without accepting its payload.

- Cancel stops the selected transfer from either side. Cancelled data cannot become a verified completed download.

- Leaving, reloading, losing the connection or closing a tab interrupts unfinished work and releases its transfer state.

- Reconnect and select the file again to restart. Automatic resume is not implemented.

- Download received files before **Clear finished files**, closing the tab or reloading. Clearing finished files releases retained receive buffers and download URLs; it is not a backup mechanism.

Confirmation dialogs protect actions that would discard active work or retained files. Read the dialog and save any completed downloads first.

Practical memory and queue limits

Limit	Default maximum
Individual file	64 MiB
Reserved and retained received payload	128 MiB
Active/queued transfers	20
Retained transfer records	100
Peers per room	2

One MiB is 1,048,576 bytes. Receive reservations include approved incoming work and completed data not yet cleared. Browser buffers and Blobs require additional memory beyond these payload figures. Lower `LANFLUX_MAX_FILE_MIB` and `LANFLUX_MAX_MEMORY_MIB` for constrained devices. This implementation does not stream large downloads directly to disk. Three retained 50 MiB files exceed the 128 MiB limit: save and clear between runs.

4. Read Network Diagnostics

Open **Network diagnostics** after pairing. Measurements come from the actual browser connection and transfer counters. **Unavailable** means the browser has not provided a reliable observation; it does not mean a measured zero.

Value	Interpretation
Signaling status	WebSocket link to Python for pairing and negotiation. It is separate from the file DataChannel.
ICE state	WebRTC connectivity establishment or failure state.

Value	Interpretation
DataChannel state	Whether the reliable browser-to-browser channel is open.
Direct P2P / TURN Relay	Classification from the selected ICE candidate pair, when reliable statistics exist. Configuring TURN alone does not prove relay use.
Connecting / Failed	Connection progress/failure. Unsupported routing statistics can remain Unavailable even when connected.
Current / average speed	File payload rate, excluding WebRTC/network overhead. Current rates use observed sampling intervals.
Total bytes / duration / ETA	Observed transfer accounting; remaining-time estimates depend on recent measured rates and may change.
Peer RTT	Explicit DataChannel ping/pong elapsed time, including browser scheduling and channel queueing; not ICMP latency.
Speed chart	A bounded history of recent measured rates, not a link-capacity prediction.

File sizes use binary MiB. Rates use decimal **MB/s** (1,000,000 bytes/second) or **Mbps** (1,000,000 bits/second); 1 MB/s equals 8 Mbps. AI throughput aggregates actual sent and received payload during its sampling interval. Sender counters count payload handed to the channel; receiver counters count accepted payload. Successful receiver verification provides the final receipt check.

Inspect connection details deliberately reveals candidate information that may include private addresses. Hide it before sharing screenshots. Public reports and AI telemetry do not include private ICE addresses. Safe connection-error summaries help investigation but do not identify a firewall's exact policy.

5. Use AI Transfer Intelligence

The existing AI Network Monitor runs two session-local scikit-learn models. It receives network metadata only and does not inspect files, change pacing or control approval.

Component	Learning requirement and output
IsolationForest	At least 30 real observations, 25 server-measured seconds and three distinct rounded feature combinations. Later samples are classified Normal or Anomaly against the frozen baseline.
RandomForestRegressor	At least 60 real, future-labeled active windows, sufficient variation and chronological training/validation coverage. Predicts approximate payload throughput over the next three seconds if activity continues.
Advisor	Explicit rules combine anomaly state, forecast direction and validation comparison. This is advisory logic, not a third trained model.

The model statuses are independent: IsolationForest can be trained while the forecast still says **Learning from network activity**. RTT probes run approximately every five seconds; one-second telemetry rows can share the latest observed RTT. Missing RTT, disconnects and invalid sampling gaps do not become invented values.

Demonstrate real training

1. Pair devices and keep both tabs foregrounded. Open Network diagnostics.

2. Send representative files in either direction and approve the offers. Continue genuine transfer activity for roughly **70-90 seconds or longer**, with real observed variation.

3. Keep each file below 64 MiB and clear saved receives before reaching 128 MiB retention. Fast transfers separated by idle time may never provide complete active windows; do not claim training in that case.

4. Once trained, observe measured Mbps, predicted next-three-second Mbps, network trend, anomaly status and advisory text during continued transfer activity.

5. Compare the model's displayed **validation MAE** with the **last-speed baseline MAE**. Lower MAE is better on this particular holdout; the model may be worse than the baseline. Neither value is an accuracy percentage or a promised speed improvement.

6. Select **Export real samples CSV** before leaving or resetting. Record the actual devices, network conditions and outcomes separately.

Anomaly scores are the negative IsolationForest decision function: positive means Anomaly; zero/negative means Normal. They are not attack probabilities. Changes in workload, CPU scheduling, hashing, congestion or idle behavior can all affect observations. Never claim an anomaly proves a cyberattack or guarantees an impending failure.

The three-second forecast disappears when idle, RTT is missing, requests fail or no fresh result arrives within three seconds. An unavailable or failed AI component must not prevent normal file sharing. **Experimental Smart Transfer Mode is not implemented**; existing transmission behavior stays the baseline.

Export and evaluate

The AI CSV contains bounded session observations and model results, without file contents, names, digests, room credentials or private ICE addresses. Export while connected, before **Reset learning baseline** or departure; models and observations are not persisted by the server. Reset clears both models for the current peer.

For the existing **IsolationForest-only** chronological evaluation CLI, collect at least 40 rows:

```
python -m aiml.evaluate /path/to/lanflux-network.csv --output network-predictions.csv
```

Windows PowerShell:

```
.\.venv\Scripts\python.exe -m aiml.evaluate "$env:USERPROFILE\Downloads\lanflux-network.csv" --output
 network-predictions.csv
```

The CLI trains on the first 30 observations and evaluates at least ten later rows. It validates the input, does not generate a dataset and refuses to overwrite an output file. RandomForest validation happens in the live monitor and its actual MAEs are exported; the CLI does not independently evaluate that model. See AI methodology and reproducible experiments.

6. Settings and browser behavior

Settings lets you change your local device label and request **Use TURN relay only** for the next connection. A host-required relay policy cannot be disabled. Select relay-only only when a relay is configured, recreate the connection, and confirm the observed route in diagnostics. A setting is not evidence that TURN worked.

Use a current desktop or mobile browser exposing WebRTC DataChannels and downloads. Chrome, Edge, Firefox and Safari are intended targets; automated browser coverage is Chromium, including narrow viewport layouts. Other engines and physical phones require their own regression checks. Local HTTP can work where WebRTC is exposed; incremental hashing does not depend on Web Crypto secure-context support. HTTPS/WSS remains required for safe internet deployment and recommended on untrusted networks.

Mobile browsers can suspend a background tab or sleeping device. Keep the tab visible and screen awake during sharing. Browser download handling varies; save into the location offered by the browser and verify the result is present before clearing LANFLUX's copy.

7. Troubleshooting

Symptom	Action
Phone cannot load the page	Use `--lan`, the computer's reachable LAN address and matching port; permit the private-network firewall rule. A guest network may isolate clients.
QR still contains localhost or a wrong address	Check the warning, configure the reachable `LANFLUX_PUBLIC_URL`, restart and create a new room. Confirm the phone can open that origin directly.
Invitation fails to join	Check the code/link, expiry and two-peer capacity. Create a fresh room if needed. An old tab may still occupy the second membership.
Website opens but connection fails	Inspect signaling, ICE and DataChannel states. Try an authorized network; ask the operator to check STUN/TURN, certificates and credentials. Page access alone does not prove peer connectivity.
College Wi-Fi blocks sharing	Public HTTPS can remove the need to reach a laptop port; configured TURN/TLS on TCP 443 may provide an allowed relay path. Protocol filtering can still block it. Ask IT for an approved configuration.
Transfer waits without progress	Check whether receiver approval or sender hashing is pending. Keep both tabs foregrounded and inspect the actual connection state.
File or memory limit reached	Use a smaller file; download and clear retained completed receives. Large-file disk streaming is unavailable.
Integrity fails	Do not use the incomplete result. Reconnect and send again; preserve the observed failure reason for diagnosis.
AI keeps learning	Wait for actual RTT and sufficient active, varied measurements. Idle time and very short files do not satisfy forecasting requirements.
Forecast or peak speed unavailable	The active window, browser observation or complete sampling interval may be missing. Do not substitute a fabricated value.
Server rejects configured origins/hosts	Use exact reachable origins and hostnames in documented configuration. Do not disable Origin checks or use wildcards.

8. Final computer-and-phone check

Create and join by QR, repeat by code/link, send and verify a file in each direction, reject one offer, cancel from each side, test several queued files and an empty file, then reload during a transfer and confirm a clean restart. Inspect diagnostics and actual AI collection, export before leaving, and save downloads before clearing. These are manual procedures to perform on your devices; the guide does not claim they were executed on every supported device.

03 / Architecture and transfer protocol

Maintained source: docs/ARCHITECTURE.md

LANFLUX has three distinct paths: a Python signaling service introduces two browsers, a WebRTC DataChannel carries file data between those browsers, and an authenticated metadata API evaluates optional network models. The application has no upload endpoint and no server-side file store.

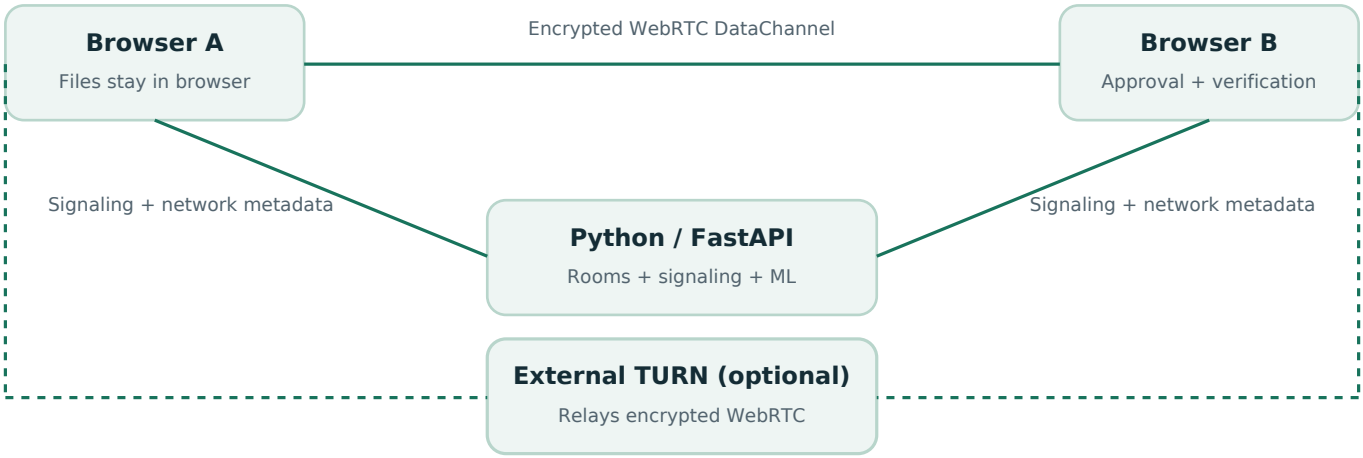
For setup and operation, see the user guide. Configuration and public deployment are covered in configuration and deployment. The security model describes what these boundaries do and do not guarantee.

Components and ownership

Component	Responsibility
`lanflux/main.py`	FastAPI application, security headers, room/QR routes, WebSocket admission, strict message parsing and static assets.
`lanflux/rooms.py`	Process-local room membership, cryptographic identifiers, pair generations, negotiation states, expiration, bounded rate-limit buckets.
`lanflux/signals.py`	Strict Pydantic schemas for offer, answer, ICE and heartbeat messages.
`lanflux/config.py`, `turn.py`	Validated environment settings and ICE configuration; optional short-lived coturn credentials.
`lanflux/network.py`	Physical-LAN address discovery and invitation-origin selection.
`lanflux/logging.py`, `__main__.py`	Sensitive-log filtering and the supported single-worker CLI.
`lanflux/static/app.js`	Existing interface, room actions, transfer cards, dialogs and navigation.
`transfer.js`	WebSocket/RTCPeerConnection lifecycle, transfer state machine, framing, approval, hashing and bounded buffering.
`metrics.js`, `diagnostics.js`, `ice.js`	Observed payload statistics, selected-candidate classification, diagnostics presentation and safe ICE configuration.
`invitation.js`, `experiments.js`	Invitation normalization and report/CSV formatting from recorded transfers.
`aiml.js`	Session-authorized telemetry collection and the AI monitor/forecast interface.
`aiml/api.py`, `model.py`, `predictor.py`	Metadata authorization, bounded IsolationForest baseline and supervised throughput forecasting.
`aiml/evaluate.py`	Offline chronological evaluation of exported anomaly-monitor observations.

All room and model state is memory-only. Run **one Uvicorn worker**. Multiple independent workers cannot share rooms or peer models, and a restart clears active sessions. Horizontal scaling would require an explicitly designed shared signaling/session layer; it is not implemented.

Data paths



TURN is a separate network relay, not a FastAPI upload fallback. File transport remains WebRTC-encrypted when a relay carries its packets. A TURN server can observe network metadata and traffic volume; it does not become the file's application receiver. A configured relay is not proof it was selected.

HTTP and WebSocket surface

Route	Purpose and boundary
`GET /, /static/...`	Browser application and bundled assets.
`GET /health`	Basic service liveness; does not prove peer connectivity.
`GET /api/config`	Public limits, STUN settings, invitation origin and TURN capability flags; no TURN passwords.
`POST /api/rooms`	Create a bounded, expiring two-peer room.
`GET /api/rooms/{id}/qr[.svg]`	SVG QR for that room's pinned invitation URL.
`WS /ws/{id}?name=...`	Same-origin admission, pairing, offer/answer/ICE and heartbeat.
`POST /api/aiml/sample`	Authorized measurement submission and model result.
`POST /api/aiml/reset`	Clear this peer's model and observations.
`POST /api/aiml/export`	Download this peer's retained metadata CSV.

The ML POST routes require a peer-bound Bearer token, strict JSON, trusted Origin, a non-expired room and two live members. There is no file-upload, remote-command or arbitrary filesystem endpoint.

Room creation and invitation

A room receives a 16-character identifier generated using `secrets.choice`. Its default absolute lifetime is one hour. The identifier is a bearer capability: possession enables admission while capacity remains available; it is not an account or proof of identity.

An explicit `LANFLUX_PUBLIC_URL` selects the invitation origin. Otherwise, an existing LAN/public page origin is retained, while a localhost invitation can use a discovered active physical private IPv4 adapter. Discovery excludes recognized Docker, virtual and VPN interfaces and prefers the default route. It is best-effort on multihomed machines: configure a reachable origin when discovery is ambiguous.

The room pins one invitation URL at creation so the QR, displayed address and Copy Link agree. Discovery refreshes for new rooms; an old room does not silently change its invitation after an adapter change. Opening a valid `?room=CODE` starts joining automatically. Invalid, expired or full rooms produce an actionable error rather than creating an unrelated room.

Pairing and signaling sequence

From	To	Message / step
First browser	Signaling server	POST /api/rooms
Signaling server	First browser	Room code, expiry, invitation URL
First browser	Signaling server	WebSocket join
Signaling server	First browser	welcome: peer ID, telemetry token, ICE settings
Second browser	Signaling server	WebSocket join from invitation
Signaling server	Second browser	welcome
Signaling server	First browser	peer-joined: new pair ID, fresh ICE settings
Signaling server	Second browser	peer-joined: new pair ID, fresh ICE settings
First browser	Signaling server	offer + pair ID
Signaling server	Second browser	validated offer
Second browser	Signaling server	answer + pair ID
Signaling server	First browser	validated answer
First browser	Second browser	ICE candidate exchange through server
First browser	Second browser	Reliable ordered DataChannel opens

The first member is the negotiation initiator. The server permits `ready -> offered -> answered`, with the offer restricted to the initiator and the answer to the other member. ICE may arrive while candidates are gathering, including before the offer reaches the server. Every paired signal carries a fresh pair ID. A departed/replaced peer's stale messages cannot negotiate a new pairing.

Clients heartbeat approximately every five seconds; default signaling inactivity closes a peer after 60 seconds. Cleanup runs every five seconds. A room that becomes empty is shortened to at most a 60-second grace period. Absolute room expiry closes its members. Departure clears the pair generation and both peers' ML state; the remaining member can pair again while the room remains valid.

The browser starts a 45-second connection deadline. It also bounds queued ICE candidates and serializes signaling processing to avoid asynchronous reordering. A failed or closed connection stops incomplete transfers and provides clean restart behavior. Automatic resumable transfer is not implemented.

File-transfer protocol, version 2

The initiator opens `lanflux-files` as an ordered DataChannel with default reliable delivery. Each browser can send and receive concurrently, while each outgoing direction uses a sequential queue. Up to one incoming file is accepted into the receive state at a time. The queue is not a multi-recipient broadcast mechanism.

Approval and integrity sequence

From	To	Message / step
Sender browser	Sender browser	Incremental SHA-256 over selected file
Sender browser	Receiver browser	file-offer(id, name, size, protocolVersion=2, sha256)
Receiver browser	Receiver browser	Validate metadata, reserve bounded memory, ask permission
		alt Receiver declines
Receiver browser	Sender browser	reject(id)
		else Receiver accepts
Receiver browser	Sender browser	accept(id)
		loop Bounded chunks with backpressure
Sender browser	Receiver browser	16-byte transfer ID + payload
Receiver browser	Receiver browser	Count bytes and incrementally hash payload
Sender browser	Receiver browser	finish(id)
Receiver browser	Receiver browser	Verify exact byte count and SHA-256
		alt Match
Receiver browser	Receiver browser	Create downloadable Blob URL
Receiver browser	Sender browser	complete(id, sha256)
		else Missing or corrupted data
Receiver browser	Sender browser	integrity-failed(id, reason)

The sender hashes in 256 KiB file slices, yielding between slices. It does not load an additional complete file solely to hash it. The receiver updates a hash as approved payload arrives. The bundled incremental implementation works on local HTTP without depending on streaming Web Crypto. Zero-byte files follow the same offer/approval/finish/digest path.

Metadata and digests travel inside the encrypted DataChannel. They are authenticated to the established WebRTC transport peer, not independently signed by a known human identity. A digest match means the receiver obtained the bytes matching the offered digest. It does not attest to the file's safety or origin.

States and terminal rules

Direction	Normal states	Other terminal outcomes
Sender	`queued -> hashing -> awaiting-approval -> transferring -> completed`	`rejected`, `cancelled`, `failed`
Receiver	`approval-needed -> transferring -> verifying -> completed`	`rejected`, `cancelled`, `failed`

A sender completes only after a valid matching receiver acknowledgement, following its finish message. A receiver completes only after exact-size and digest checks. Terminal transitions are guarded so late cancellation, duplicate acknowledgements and delayed frames do not convert a cancelled/failed transfer into success. Downloads require completed, verified receive state and a valid Blob URL.

Unknown control types/fields, invalid IDs, unapproved payload and impossible transitions are rejected. Control frames are bounded to 8,192 string characters. Binary frames contain a 16-byte transfer identifier and at most 16 KiB of payload. The sender reduces payload size when a finite negotiated SCTP message limit requires it; impossibly small limits fail cleanly. A chunk-count bound rejects excessive tiny-buffer overhead.

Backpressure and timeouts

`bufferedAmountLowThreshold` is 64 KiB. When the channel's buffered amount exceeds 256 KiB, sending waits for a low-buffer event, closure, cancellation or a 30-second stall timeout. The implementation rechecks after registering listeners to avoid a missed drain event. The high-water mark is a sending threshold, not a promise that all browser memory is limited to that exact value.

Approval has a 120-second inactivity timeout; active preparation/transfer/verification has a 90-second inactivity timeout, checked by the five-second maintenance tick. These are inactivity thresholds, not maximum total durations. Both sides can cancel. Disconnects close the channel/peer connection, fail incomplete work, discard incomplete chunks and stop related listeners/waiters. Leaving clears periodic timers and room credentials.

Memory and lifecycle

Default limits are 64 MiB per file, 128 MiB of reserved/retained receive payload, 20 nonterminal/queued entries and 100 retained transfer records. Lower file and payload budgets are configurable. A receive reserves its announced size before acceptance; rejected/cancelled/failed transfers release it. Completed receives continue to consume the budget until cleared. Clearing terminal records revokes Blob URLs and releases their reservations.

These limits bound application payload accounting, not total browser resident memory. ArrayBuffer/Blob copies, hash state, DOM and browser networking add overhead. Mobile users should clear completed downloads and keep foreground tabs awake. No streaming-to-disk, folder traversal, persistent transfer archive or automatic resume is advertised.

Observability and units

`PayloadMetrics` uses monotonic browser time and actual application payload-byte counters. Current speed is a sample delta divided by elapsed time; average is bytes divided by active transfer duration; ETA uses remaining bytes and current positive speed. With concurrent bidirectional transfers, aggregate active duration is union time, not the sum of overlapping send and receive durations. Speed history retains at most 60 samples.

File sizes use binary MiB (`1 MiB = 1,048,576 bytes`). Transfer UI rates may be formatted in decimal MB/s; AI throughput uses decimal Mbps (`bytes x 8 / seconds / 1,000,000`). These are payload rates, excluding protocol overhead. Sender counters measure payload handed to the DataChannel; receiver counters measure accepted payload. A successful verified transfer is the end-to-end correctness check.

RTT is explicit DataChannel ping/pong elapsed time, approximately every five seconds, not ICMP or pure network propagation latency. Channel queues and scheduling contribute. Reliable selected-pair ICE statistics classify direct versus relay only when both candidate types are available and unambiguous. Unsupported metrics show Unavailable. Private ICE addresses require the explicit inspection action and never enter ML metadata.

Session-local intelligence

The anomaly model and forecaster consume the same authorized observations but solve different problems:

Model	Input and training	Output
IsolationForest	`log1p(RTT)`, `log1p(Mbps)`, active flag; at least 30 real rows, 25 server-monotonic seconds and three distinct feature combinations. Up to 120 baseline rows, 64 trees, one worker. Baseline freezes after training.	Normal/Anomaly and negative decision-function score for later observations.
RandomForestRegressor	Five past active samples: speed level/mean/variation/trend, RTT/variation, active fraction and observed route flags. Labels are subsequent measured three-second weighted throughput. At least 60 labeled windows and meaningful speed variation; bounded 48-tree forest.	Experimental next-three-second aggregate active-throughput forecast and chronological holdout MAE versus last-speed baseline.

The forecaster accepts 750-2000 ms intervals with consistent server receipt gaps. Missing/irregular samples break pending windows. It excludes idle input/label windows; real active stalls can have zero speed. The earliest 70% of examples are training candidates and the latest 30% validation; training labels overlapping validation inputs are purged. At least 30 training and 12 validation examples must remain. At most 240 labeled examples are retained; a successful refit needs 30 new labels and 60 seconds since the previous fit.

Recommendations are transparent advisory rules, not a third trained model. They combine anomaly status, forecast trend and validation comparison. Forecasts clear when stale, idle or unavailable. No AI decision paces, blocks, cancels or alters transfers; Experimental Smart Mode is deliberately absent.

Model operations run through `asyncio.to_thread`, with two concurrent operations and per-peer locks. There are at most 32 monitor instances and 600 retained CSV records per monitor. Models and records disappear on reset, pair change, departure, expiry or server restart. Measurements are authorized browser reports, not independent instrumentation or fabricated training data. See AI algorithms, evaluation and demonstration for exact interpretation and limitations.

04 / Machine learning and transfer intelligence

Maintained source: docs/AI_MONITOR.md

LANFLUX combines experimental IsolationForest anomaly detection with a supervised RandomForest throughput forecast, reusing existing WebRTC diagnostics. It does not inspect files, modify transfer messages, throttle transfers or claim to detect attacks reliably.

Observations

Connected browsers sample approximately once per second:

Field	Source	Used by IsolationForest
RTT, milliseconds	Explicit DataChannel ping/pong elapsed time	Yes, `log1p`
Payload throughput, decimal Mbps	Actual sent-plus-received file-byte counter delta / elapsed sample time	Yes, `log1p`
Active transfer	Transfer-engine `transferring` state or observed payload in the interval, including short completed transfers and stalled active intervals	Yes, boolean
Cumulative bytes, interval and observed route	Transfer counters, clock and selected ICE candidate statistics	Export context only

RTT probes run approximately every five seconds, so adjacent one-second rows may share the latest observation. Browser scheduling and channel queueing affect RTT; this is not ICMP latency. Sender counts indicate bytes handed to the channel, receiver counts indicate accepted payload. A measured idle interval can legitimately have zero throughput. Missing RTT, disconnected sessions, counter resets and sampling gaps over ten seconds are skipped, never replaced with fabricated values.

IsolationForest training and prediction

Each peer has its own session-local monitor. Training requires at least **30 actual observations**, at least **25 seconds measured by the server's monotonic clock**, and **three distinct rounded feature combinations**. The UI stays **Collecting training data** when any requirement is missing; constant telemetry explains that more variation is needed.

`aiml/model.py` fits scikit-learn `IsolationForest` with 64 trees, `contamination='auto'`, a fixed reproducibility seed, one worker and at most 120 baseline rows. The baseline then freezes. The observation completing training is not presented as a holdout prediction; later observations are evaluated against the baseline.

Prediction `-1` becomes **Anomaly**, `1` becomes **Normal**. The displayed score is the negative decision function: positive means anomaly, zero/negative means normal. It is not a probability or confidence percentage. The fixed seed makes model fitting reproducible; it does not generate training observations.

Reset deletes the current peer's model and records. Pair replacement, departure, room expiry and server restart clear session data. Models and CSV rows are never persisted server-side. Retention is bounded to 600 observations per monitor, 32 monitors per server and two concurrent model operations.

Metadata endpoint access

`POST /api/aiml/sample`, `/api/aiml/reset` and `/api/aiml/export` require a per-peer bearer token delivered through the authorized room WebSocket, a trusted browser Origin, a non-expired room and both live peer memberships. Tokens stay in browser memory, outside invitation links and localStorage.

Strict schemas reject invalid/nonfinite numbers, unrelated fields and malformed JSON. Requests are capped at 2 KiB before complete buffering, paced and rate-limited to 180/IP/minute and 90/peer/minute. Per-peer locks serialize operations; model work runs off the async event loop. File bytes, filenames, hashes, private ICE addresses and room credentials are absent from telemetry exports.

Authorized clients can falsify telemetry. HTTPS/WSS protects the transport on untrusted networks; these measurements are browser reports, not independently attested observations.

Real-data CSV and evaluation

Choose **Export real samples CSV** while connected. The download includes UTC server receipt time, measured fields, baseline/evaluation phase, model state, prediction and score. Export before resetting or disconnecting.

Collect at least 40 rows, then evaluate locally:

```
python -m aiml.evaluate /path/to/lanflux-network.csv --output network-predictions.csv
```

On Windows PowerShell:

```
.\.venv\Scripts\python.exe -m aiml.evaluate "$env:USERPROFILE\Downloads\lanflux-network.csv" --output
 network-predictions.csv
```

The first 30 chronological rows train the evaluation baseline; at least ten later rows form the holdout. `--train-samples 60` requires at least 70 rows. Inputs are strictly validated, bounded to 600 rows/2 MiB, and must include monotonic cumulative byte counts and valid timestamps. The CLI does not generate data or overwrite an existing output file.

The human-readable report states baseline/holdout counts, mean observed RTT/throughput, score range and actual anomaly count; the output CSV contains holdout predictions and scores. Evaluation can differ from the live model if older rows have rolled out of bounded retention. Accuracy/precision/recall remain unavailable without independently verified labels. Unit tests use explicitly synthetic fixtures for algorithm correctness; production and browser evaluation never use those fixtures as observations.

Demonstration and limitations

Pair the computer and phone, keep both foregrounded, open Network diagnostics, and send representative files during collection. Wait at least 30-45 seconds; insufficient variation or missing RTT can take longer. After training, observe the score and prediction during later transfers or a real change in distance/congestion. Export all outcomes, including normal ones. An anomaly is not guaranteed and should not be manufactured for a presentation.

Thirty rows are an MVP threshold, not proof of statistical adequacy or 30 independent latency probes. Workload changes, file hashing, CPU scheduling, idle time and Wi-Fi variation can produce false positives; the model can miss anomalies. No labeled accuracy, Wi-Fi benchmark or physical-device performance claim is supplied. Stronger evaluation needs longer representative sessions and independent ground-truth annotation.

Three-second throughput forecast and transfer advice

`aiml/predictor.py` adds a supervised **RandomForestRegressor** alongside IsolationForest. It predicts aggregate sent-plus-received payload Mbps over approximately the next three seconds **if transfer activity continues**, not link capacity or a specific file's completion speed. Five past observations provide last/mean/standard-deviation throughput, throughput slope, last RTT/RTT variation, active fraction and reliable direct/relay indicators. Unknown routes are not guessed.

An input window is anchored before its label exists. The following measured intervals supply the duration-weighted three-second target; the final interval is proportionally clipped at the horizon. This is an interval-rate approximation, not sub-second ground truth. Only fully active input and future windows contribute labels; stalled active intervals may have real zero throughput. Idle periods cannot manufacture a good evaluation. Samples outside 750-2000 ms, inconsistent receipt gaps, missing observations and disconnections break pending windows.

Training requires **60 genuinely labeled windows**, at least 30 training and 12 validation examples after purging, six distinct recent-speed values, and meaningful throughput variation. The first 70% of windows are training candidates; later windows validate. Training examples whose labels overlap the first validation input window are removed. Validation MAE in Mbps is displayed beside the MAE of always predicting the last observed speed. These are measured holdout errors, not accuracy percentages or proof of improvement. Overlapping windows remain correlated, so this small validation is not an independent network benchmark.

The target and validation calculations are explicit:

```
future Mbps = sum(observed interval Mbps * included interval seconds) / 3
model MAE = mean(abs(actual future Mbps - RandomForest prediction Mbps))
baseline MAE = mean(abs(actual future Mbps - last observed Mbps))
```

For example, an anchor's five input rates are already fixed before any later observation arrives. Only the next intervals can create its target; changing those later rates cannot change the stored input features. The purge criterion is `training.label_end < first_validation.input_start`. The fitted model uses the training fold only, so validation rows are not subsequently folded back into the reported live model. This protects the stated holdout comparison; it does not make correlated rolling windows independent.

The forest uses 48 trees, depth at most six, minimum leaf size three and one worker. At most 240 labeled examples, five history points and eight pending anchors are retained per existing session monitor. Refitting needs 30 new labels and at least 60 seconds since the last fit. Fitting/prediction uses the existing off-event-loop, authenticated telemetry request path; no new service, dependency, model download or file metadata is needed. Reset/session cleanup clears both models. CSV exports append actual forecasts, validation MAEs and trend; the existing `aiml.evaluate` CLI still evaluates IsolationForest only.

AI Transfer Intelligence shows measured speed alongside a fresh forecast, learning/trained status, trend and advisory text. Recommendations are explicitly **rules**, combining IsolationForest status, forecast direction and the measured validation comparison. Trend uses a change threshold of the larger of 0.25 Mbps or 15% of measured speed. Advice flags experimental forecasts that have not beaten the last-speed baseline. An anomaly is not proof of an attack. Forecasts disappear during idle activity, missing latency, request failure or after three seconds without a fresh result; normal file sharing remains independent.

Demonstrate

1. Start `python -m lanflux --lan --port 8000`, pair the devices and open Network diagnostics.

2. Keep tabs foregrounded and send representative files while approving incoming offers. Collect at least roughly 70-90 seconds of **active**, varied measurements; idle time does not count. Several short transfers may not supply complete five-second inputs plus three-second labels. Use files within the existing 64 MiB limit and clear saved receives before hitting 128 MiB retention.

3. Wait for the separate RandomForest status to become trained. During continued transfers, observe measured/predicted Mbps, trend, model MAE and baseline MAE. Predictions may be worse than the baseline; report that result honestly.

4. Export the real CSV before resetting/disconnecting. No synthetic test fixtures are used in live training. Fast or short sessions may legitimately remain "Learning from network activity".

Experimental Smart Transfer Mode is deliberately not implemented: no pacing, chunk sizes, negotiated limits, backpressure or wire protocol are changed. A controlled performance comparison is needed before letting uncertain forecasts influence transmission.

Implementation and test map

File	Responsibility
`aiml/model.py`	Strict observations, anomaly baseline, session records and forecaster/advisor composition
`aiml/predictor.py`	Past-only features, delayed labels, bounded forest fitting, chronological validation and advisory rules
`aiml/api.py`	Existing peer authorization, request caps, per-peer serialization and off-event-loop operations
`lanflux/static/aiml.js`	Actual browser sampling, authenticated requests, fresh/unavailable presentation and CSV/reset controls
`tests/test_predictor.py`	Synthetic correctness fixtures for leakage, alignment, timing gaps, eligibility, fitting, evaluation, advisor and failure fallback
`tests/test_aiml.py`, `tests/test_aiml.mjs`	API authorization, strict metadata, sampler calculations and forecast freshness
`tests/browser.test.mjs`	Actual browser observations, approved verified transfers, model training and displayed predictions

Run `python -m pytest -q tests/test_predictor.py tests/test_aiml.py`, `npm run test:unit` and `npm run test:e2e`. Full executed results and limits are recorded in verification. Synthetic unit-test fixtures never seed a live session.

05 / Security and privacy

Maintained source: docs/SECURITY.md

LANFLUX is a two-browser file-sharing application with a small signaling and metadata service. Its controls limit unintended access and resource use while preserving local-LAN operation. They do not authenticate a person's real identity, inspect file safety or guarantee connectivity on a restricted network.

For operational setup and hardened examples, read configuration and deployment. For message flows and implementation ownership, see architecture.

Trust boundaries

Boundary	What crosses it	What must be trusted
Browser <-> FastAPI	Room capability, display name, SDP/ICE, heartbeat, peer-bound telemetry authorization.	The served application and signaling service; HTTPS/WSS for untrusted networks.
Browser <-> peer	Approved file metadata, SHA-256, encrypted file chunks and diagnostic ping/pong.	The paired peer for offered content and its stated identity.
Browser <-> STUN	Address-discovery traffic.	The configured discovery operator can observe connection metadata.
Browser <-> TURN	WebRTC-encrypted relayed traffic and temporary allocation credentials.	The relay operator for availability and network metadata handling.
Browser <-> ML API	Numeric network observations and categorical route; model results/CSV.	The authenticated reporting browser; reports are not independently attested.

FastAPI has no file-upload or file-content storage path. Filenames, digests and payload are exchanged through the DataChannel. This statement concerns the supplied application; separately configured proxies, extensions and host software have their own behavior and retention.

Transport and identity

WebRTC DataChannels use the browser's encrypted transport. Direct P2P means no TURN relay was observed in the selected candidate pair; it does not mean the devices are on the same Wi-Fi or that no routers exist between them. TURN Relay means a relay candidate was actually selected. Encrypted file packets passing through TURN do not become plaintext uploads to FastAPI.

Local HTTP remains supported for trusted development/LAN use. HTTP does **not** protect the served JavaScript or signaling from interception/modification. Use HTTPS/WSS for public or untrusted networks. WebRTC encryption does not compensate for an attacker replacing the application or manipulating unauthenticated signaling.

A room code is a bearer capability. Anyone with it can occupy an available room slot. The application does not provide accounts, a second-factor room password, a human identity certificate or out-of-band fingerprint verification. Verify the other device through a trusted conversation and share invitations privately. The second-peer capacity check prevents a third participant from joining that room, but it cannot identify whether the first two participants were the intended people.

Room admission and authorization

Room IDs use cryptographically secure generation and a restricted 16-character alphabet. Peer IDs, transfer IDs, telemetry tokens and new pair generations are also randomly generated. Default room life is one hour, with periodic expiration and at most 60 seconds of empty-room grace after both members leave.

WebSocket admission validates a single trusted Host and requires one canonical Origin matching the current website or an explicitly allowed exact origin. Missing, null, malformed, duplicate or untrusted browser Origins are rejected. Trusted public/custom hosts and allowed origins are separately configurable; accepting a Host does not authorize arbitrary cross-origin pages. Localhost, supported private addresses and LAN names remain usable without a wildcard public policy.

Origin checks protect against ordinary cross-site browser connections. A non-browser client can forge an Origin header, so Origin alone is not user authentication or a defense against a client that already possesses a room code. Public deployments need additional edge admission controls and abuse monitoring.

The server binds each admitted socket to a specific peer object. It selects the other member itself rather than accepting an arbitrary client-supplied recipient. Paired messages must carry the current pair ID, use valid negotiation roles and follow `ready -> offered -> answered`. Replacement/departed peers cannot reuse a stale generation to negotiate the next pair.

Strict messages and resource bounds

Signaling accepts text JSON only, rejects duplicate object keys and validates strict typed schemas with no extra fields. The application caps signaling at 64 KiB UTF-8, SDP at 49,152 characters and individual ICE candidate strings at 4,096 characters. ICE can precede offer forwarding because browser candidate gathering is asynchronous; other negotiation transitions remain checked.

Default bound	Purpose
Two peers per room	Pairing capacity.
1,000 rooms and 512 peers	Bounded process-local admission.
10 room creations/IP/minute	Room-creation abuse limit.
30 join attempts/IP/minute	WebSocket admission pacing.
60 QR requests/IP/minute	QR rendering bound.
180 signaling messages/peer/minute	Socket message pacing.
64 MiB/file, 128 MiB retained/reserved receive payload	Browser payload memory budget.
20 nonterminal/queued files, 100 retained records	Browser queue/history bound.
60 chart samples	Bounded diagnostics history.
2 KiB/ML request, 180 requests/IP/minute, 90 requests/peer/minute	Metadata API pacing and body bound.
32 monitors, 600 retained rows/monitor, two model operations at a time	Model memory/CPU bounds.

IP rate buckets themselves are bounded. These in-process controls are useful MVP protections, not a distributed denial-of-service system. Shared college NATs can make several legitimate users share an IP limit. Large-scale/public operation needs edge request limits, OS/service hardening, bandwidth quotas and monitoring.

Receiver consent and file integrity

Every incoming file requires explicit approval. The receiver reserves its advertised size before accepting and refuses offers exceeding queue, file or receive-payload limits. It rejects payload before permission, invalid IDs, unknown controls, unexpected state transitions, excess declared bytes and excessive tiny chunks. Outgoing payload is chunked with negotiated-message-size checks and `bufferedAmount` backpressure.

The sender computes SHA-256 incrementally over the original file. The receiver computes SHA-256 over actual accepted payload and requires the exact declared byte count. Only matching, completed files receive a download Blob URL. A sender also waits for a valid matching completion digest. Missing bytes, corruption, unavailable hashing, cancellation and connection failure cannot produce a verified successful state.

Integrity Verified means transferred bytes match the peer's digest. An intentional malicious sender can offer dangerous content and its correct hash. SHA-256 verification does not establish real-world identity, perform malware scanning, make a file safe to open or provide nonrepudiation. A bare digest is not an independently signed identity claim.

The incremental hash implementation is bundled locally with its third-party license/provenance notices. It avoids a second complete file-sized hashing buffer and remains usable without `crypto.subtle` on local HTTP. No third-party model or JavaScript package is silently downloaded at runtime.

Filenames, HTML and browser cleanup

Incoming filenames are treated as untrusted text. The transfer layer removes path separators, control/bidirectional characters and unsafe filename characters, bounds length, handles Windows reserved names and avoids trailing dot/space edge cases. Interface rendering uses text rather than interpreting filenames as markup. These names do not become server filesystem paths: the backend never saves the file.

Downloaded content uses an `application/octet-stream` Blob with a download filename. A user still chooses whether to open it using other software. Keep normal browser/operating-system file handling protections enabled.

Security headers disable framing, sniffing and unnecessary device permissions; CSP restricts application scripts/styles to its own origin and prevents embedded objects/base-URL replacement. CSP permits WebSocket connection schemes for signaling, so it should not be described as an absolute network-exfiltration barrier.

Incomplete transfer cleanup discards chunks, hash instances and file references and releases reserved receive payload. Leaving/disconnect closes WebRTC objects and drains buffer waiters; leave stops heartbeat/statistics timers. Completed receives remain deliberately retained for download until cleared; clearing revokes Blob URLs and releases their accounted payload. Browser allocation/Blob overhead can exceed the advertised payload budget, so the limit is not a total-RAM guarantee.

ML authorization, privacy and failure isolation

The admitted WebSocket supplies a fresh per-peer telemetry Bearer token. Tokens stay in browser memory and are absent from invitation URLs and localStorage. Each metadata request also requires a trusted Origin, a non-expired room, the matching peer and two live members. Strict models reject unrelated fields and invalid/nonfinite values; body limits apply while streaming the request before complete buffering. Per-peer locks prevent overlapping model operations.

Models are separate for each reporting peer. Pair changes, departure, reset, expiry and restart clear them. Retained CSV rows contain observations, receipt timestamps, anomaly results, forecast values and validation errors. They exclude file contents, filenames, file hashes, private ICE addresses, room codes and authorization tokens. An exported CSV is a user-created local artifact; share it only with appropriate context

and permission.

Authorized clients can submit misleading telemetry. The models are experimental, not adversarially robust attack detectors. IsolationForest outliers can reflect workload/CPU/browser scheduling changes; a forecast can be worse than the last-observed-speed baseline. Genuine future measurements are needed to label forecast examples, with chronological purging before holdout evaluation. Synthetic fixtures are limited to tests, and no idle-data success claim is substituted for active-transfer forecasting.

Model fitting and inference run off the async event loop under bounded concurrency. The frontend clears stale forecasts and treats API/model failures as unavailable. AI predictions do not change the file protocol, approval, integrity rules, chunk size or pacing. Sharing remains independent of AI availability.

TURN credential handling

Public configuration exposes STUN and relay capability flags, not TURN passwords. Admitted peers receive fresh private ICE configuration in `welcome` and `peer-joined`, including replacement pairings. Browser-held relay credentials are usable by that browser's user; secrecy from an admitted user is not claimed.

Recommended coturn REST mode keeps a shared secret only on Python and the TURN service, producing timestamped per-peer usernames and Base64 HMAC-SHA1 credentials. The default issued lifetime is 600 seconds. HMAC-SHA1 here is coturn's credential protocol; it is distinct from file SHA-256 integrity verification. Credential expiry restricts new allocations; existing allocations may continue under their session credentials. Keep clocks synchronized. Static relay username/password configuration is available for controlled deployments but lacks the same short lifetime.

The shared secret must be at least 32 characters. Empty/inconsistent credentials, duplicate/invalid ICE URLs and unsupported relay-only configuration fail validation. Frontend configuration discards relay credentials on leave/pair loss and rejects expired credentials for a new connection. Do not log credentials or commit `.env`, certificate keys, TURN databases or generated datasets.

Reverse proxy and restricted networks

The supported CLI binds loopback by default. `--lan` explicitly opens LAN binding. For public deployment, terminate HTTPS at a maintained proxy, keep Uvicorn private, run one worker and trust forwarded headers only from exact configured proxy IPs. The CLI rejects blanket wildcard/CIDR proxy trust. Preserve the external Host and WebSocket Upgrade, and configure the public origin consistently.

The supplied Nginx example uses Basic authentication for the website and signaling admission. Exactly `/api/aiml/sample`, `/api/aiml/reset` and `/api/aiml/export` use the backend's strong peer Bearer authentication instead, because Basic and Bearer occupy the same Authorization header. Do not broaden this exception to all APIs or WebSockets. A public operator must configure real access credentials, TLS and edge limits rather than treating example files as a deployed service.

A coturn TLS/TCP 443 listener needs its own public endpoint, typically a separate host/IP from the website's TCP 443. TURN/TLS is not HTTP and cannot be passed through an ordinary HTTPS location proxy. Relay UDP ports, certificates, allocation quotas, private/special peer-address denials and host firewalls must also be configured. Never enable an open unauthenticated relay.

Direct P2P remains available under the default ICE `all` policy; native ICE evaluates permitted candidates. Relay-only is explicit and requires configured TURN. No routing workaround bypasses a college's policies. A firewall can still block TURN/TLS on 443 or enforce client isolation. Ask the network administrator for an approved path. Actual connectivity requires externally supplied infrastructure and cannot be guaranteed by local code alone.

Logs, retained data and remaining limitations

The supported startup disables access logging, filters bearer room paths/query strings and suppresses protocol debug frame bodies. Reverse proxies and custom instrumentation need their own redaction. Keep signaling payloads, tokens, private candidate addresses and room URLs out of production logs. Explicit local ICE inspection may reveal addresses to the user; it is not sent to AI or public reports by default.

The service has no account management, independent peer-identity verification, malware scanning, durable audit log, distributed rate limiter or automatic transfer recovery. Room URLs may remain in a user's browser history or be shared outside the app. An authorized hostile peer can still waste resources within/around client controls. Dependency/runtime patching, trusted hosting, endpoint security and careful handling of received files remain operator/user responsibilities.

Report a vulnerability privately to the repository maintainer before publishing exploit details; do not include credentials, private room URLs or unredacted user data in public issues.

06 / Configuration reference

Maintained source: docs/CONFIGURATION.md

Copy `.env.example` to `.env`; `python -m lanflux` loads it automatically. Alternatively pass `--env-file PATH`. Run `python -m lanflux --help` for binding, port and TLS options. Keep real credentials and private keys out of Git.

Environment variables

Variable	Purpose / default
`LANFLUX_PUBLIC_URL`	Explicit HTTP(S) invitation origin, without a path. Empty uses physical-LAN discovery for localhost, otherwise the page origin.
`LANFLUX_ALLOWED_ORIGINS`	Additional exact HTTP(S) browser origins, comma-separated, without paths or wildcards. Same-origin access works by default.
`LANFLUX_TRUSTED_HOSTS`	Additional exact public/custom DNS hostnames, without ports or wildcards. Hosts from PUBLIC_URL and ALLOWED_ORIGINS are also trusted.
`LANFLUX_ROOM_TTL_SECONDS`	Absolute room life: default 3600; range 60-86400.
`LANFLUX_PEER_IDLE_TIMEOUT_SECONDS`	Idle signaling cleanup: default 60; range 15-300. Clients heartbeat every five seconds.
`LANFLUX_MAX_ROOMS`, `LANFLUX_MAX_PEERS`	In-process resource caps: defaults 1000 and 512.
`LANFLUX_MAX_FILE_MIB`, `LANFLUX_MAX_MEMORY_MIB`	Browser payload limits: maximum/default 64 and 128 MiB. Lower them for constrained phones.
`LANFLUX_STUN_URLS`	Comma-separated STUN URLs; Google STUN defaults. Empty disables STUN for isolated-LAN tests.
`LANFLUX_TURN_URLS`	Comma-separated `turn:`/`turns:` URLs; requires either the shared secret or both static credentials below.
`LANFLUX_TURN_SHARED_SECRET`	Server-only coturn REST authentication secret, 32-4096 characters. Recommended for public deployments; must match coturn's `static-auth-secret`. Mutually exclusive with static username/credential.
`LANFLUX_TURN_CREDENTIAL_TTL_SECONDS`	Lifetime of newly issued ephemeral TURN credentials: default 600; range 60-3600 seconds.
`LANFLUX_TURN_USERNAME`, `LANFLUX_TURN_CREDENTIAL`	Alternative static browser-visible relay credentials for controlled private/admin deployments. Never commit actual values; leave both empty in shared-secret mode.
`LANFLUX_ICE_TRANSPORT_POLICY`	`all` (default) or `relay`; relay-only requires a reachable configured TURN service.

Local Wi-Fi

The default CLI binds `127.0.0.1`. Explicitly enable LAN access:

```
python -m lanflux --lan --port 8000
```

Open `http://localhost:8000` on the computer and create a room. The invitation chooses an active physical adapter's private IPv4 address, preferring the default route. Docker, virtual and VPN adapters are excluded. Detection refreshes every 30 seconds for new rooms; each existing room retains one consistent QR/copy/display invitation.

On systems with multiple physical networks or unavailable adapter discovery, set `LANFLUX_PUBLIC_URL` to the actual reachable scheme, hostname/address and port. Restart and create a new room. This changes invitations, not the server bind address. Opening the computer's LAN origin directly also preserves that origin. Find addresses with `ipconfig` on Windows, `ip -4 addr` on Linux, or `ipconfig getifaddr en0` on macOS (adapter names vary).

Allow the selected port on private-network firewall rules. Avoid guest-network client isolation. Keep both browser tabs foregrounded; sleeping/backgrounded mobile browsers may suspend WebRTC and heartbeats.

Origin and Host controls

Browser WebSocket and AI requests require a canonical same-origin match or an explicitly configured trusted origin, including scheme and port. Missing/null/malformed/duplicate Origins are rejected. Adding a trusted Host does not independently permit a cross-origin browser.

Host validation accepts localhost, private/loopback IPs, unqualified LAN names and `.local`. Public/custom DNS names need explicit trust. Wildcard public Hosts and arbitrary cross-site connections are rejected.

HTTPS and internet deployment

Use HTTPS/WSS on untrusted networks. Trusted local HTTP may expose WebRTC, but HTTP signaling can be intercepted or modified and does not authenticate the sending person. The hash implementation itself works without Web Crypto or a secure context.

Run **one Python worker**: room membership, signaling and AI models are process-local. Put a TLS reverse proxy in front of a private Uvicorn upstream, forward WebSocket Upgrade, and preserve the browser-visible Host. Configure the exact public origin and avoid caching `/api/config`. Trust forwarded headers only from your own proxy; do not expose an upstream with blanket forwarded-IP trust. Proxy logs should redact invitation queries and room socket paths.

The supported CLI accepts `--forwarded-allow-ips 127.0.0.1,::1` to trust exact proxy IPs; wildcards and CIDR ranges are rejected. Use `--no-proxy-headers` when no trusted proxy supplies these headers. The deployment example binds Python to `127.0.0.1:8000`, with Nginx on the same host. Do not publish port 8000 or add arbitrary public clients to the proxy trust list. See the complete HTTPS, coturn and systemd deployment guide.

For local TLS with certificates trusted by both devices:

```
python -m lanflux --lan --ssl-certfile PATH --ssl-keyfile PATH
```

The development server is not exposed publicly by default. A public deployment also needs edge abuse controls and appropriate operating-system/server maintenance.

Optional TURN

For a separately operated coturn service, use ephemeral credentials. Example placeholders, not operational credentials:

```
LANFLUX_TURN_URLS=turns:turn.example.com:443?transport=tcp
# Required: fill with a private random secret, >=32 characters; match coturn.
LANFLUX_TURN_SHARED_SECRET=
LANFLUX_TURN_CREDENTIAL_TTL_SECONDS=600
LANFLUX_TURN_USERNAME=
LANFLUX_TURN_CREDENTIAL=
```

The shared secret remains on the Python and coturn servers. `/api/config` exposes STUN and TURN availability flags, without TURN passwords. After room admission, `welcome` and each `peer-joined` WebSocket message deliver fresh per-peer TURN credentials and their expiration. LANFLUX uses coturn's timestamped username and Base64 HMAC-SHA1 credential format. Credentials reaching a browser are usable until their expiry; never treat them as private from that browser's user. Existing allocations can retain their session credentials; expiry primarily prevents new allocations with old credentials. Replacement peers receive fresh credentials. Keep both server clocks synchronized. Coturn REST authentication reference.

Room creation is anonymous, so admission is not account authentication. A public operator must apply edge access control, request limits, coturn allocation/bandwidth quotas and monitoring. The deployment examples include a password-protected Nginx edge and a separate TURN host. No TURN service is bundled or automatically provisioned.

With the default `all` policy, the browser's native ICE agent evaluates permitted host, STUN and TURN candidates; TURN is available without an application-level reconnect loop. This does not guarantee a specific selection order or connectivity on every network. `relay` permits only relay candidates. WebRTC ICE transport policy.

TURN over TLS on TCP 443 is TURN traffic, not HTTPS. It may still be blocked by a college firewall, protocol inspection or allowlist. Both devices must reach the public website and TURN endpoint, and the TURN host needs working relay UDP ports. Ask the network administrator for an authorized path when policy blocks the service.

Settings -> Relay only applies to the next connection. Confirm **TURN relay** from the observed selected ICE pair before labeling an experiment as relayed. A configured relay is not proof that it was used.

Security and resource boundaries

The server never receives or stores file contents. Room identifiers are cryptographically generated 16-character bearer capabilities; share invitations privately. Rooms have two peers, absolute expiration, disconnected-peer cleanup and fresh pairing IDs. Signaling validates message sizes, fields and negotiation transitions. Default per-client limits are room creation 10/minute, joins 30/minute and QR requests 60/minute; signaling is 180/peer/minute. State tables are bounded.

Files require receiver approval and exact byte-count/SHA-256 verification. Filenames are normalized and rendered as text. No upload, arbitrary filesystem or remote-command routes exist. Completed receives count toward the 128 MiB payload limit until cleared; browser buffers/Blobs add overhead beyond this limit. No streaming-to-disk or automatic resume is advertised.

The recommended CLI disables access logging and filters sensitive WebSocket payloads and room credentials. A matching hash proves transferred-byte integrity relative to the peer's metadata, not identity or file safety. An authorized malicious peer can still send unwanted content or cause denial of service.

07 / HTTPS, TURN and public deployment

Maintained source: docs/DEPLOYMENT.md

This guide describes infrastructure you must operate; the repository does not provision a VPS, DNS, certificates or a TURN service. It is suitable for an authorized small deployment, including testing from Wi-Fi with client isolation. It does not guarantee passage through restricted networks or override college policy. Obtain permission from the network administrator when required.

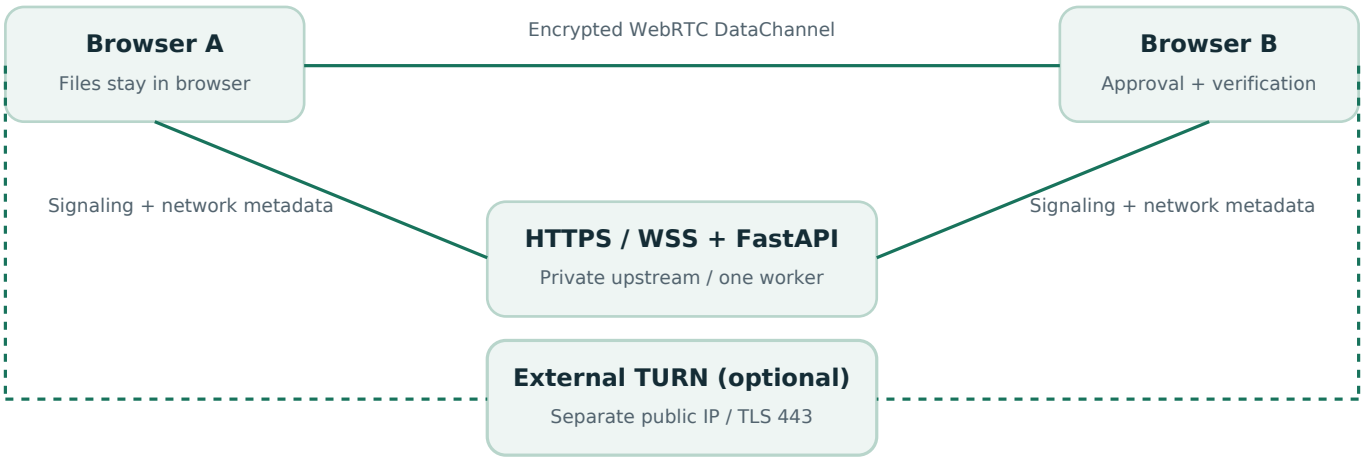
Topology and prerequisites

Use a supported Linux server with Python 3.10+, Nginx, systemd and a current security-patched coturn package. You need two public IP endpoints, DNS you control, valid publicly trusted certificates, provider firewall access and sufficient relay bandwidth. These examples use:

Endpoint	Public listener	Service
`share.example.com` -> web server IP	TCP 443	Nginx HTTPS and same-origin WSS
Web server loopback	`127.0.0.1:8000` only	One LANFLUX Python process
`turn.example.com` -> separate TURN server IP	TCP 443	coturn TURN over TLS
TURN server IP	UDP 49160-49259	TURN relay allocations

The two hostnames must resolve to **different public IPs** when both listeners bind TCP 443. Two DNS records on one IP do not avoid a port conflict. Two VPSs are the simplest arrangement; one server with two explicitly bound public IPs also works after adapting the Nginx listener. Nginx's HTTP `proxy_pass` cannot forward raw TURN traffic. This guide does not implement a shared-port TCP multiplexer.

Use DNS-only records for TURN; a normal HTTP CDN proxy does not carry TURN. Publish AAAA records only after configuring and testing IPv6 routing, listeners, certificates and relay firewall rules. The supplied coturn example binds an IPv4 address. A cloud public IP implemented by 1:1 NAT needs correct `external-ip=PUBLIC_IP/PRIVATE_IP`, binding to the local private address, with port-preserving forwarding of the complete relay range. A directly assigned public IP is easier to diagnose. Coturn server options.



File bytes use WebRTC's encrypted transport. A selected TURN route relays encrypted file traffic through coturn; it is not a direct network path between browsers. FastAPI handles signaling and does not upload or store files.

Firewall requirements

Configure both the provider security group and the operating-system firewall. Permit return traffic for established connections. Restrict administrative SSH to administrator addresses.

Location	Required rule for these examples
Client network outbound	TCP 443 to the web domain and separately TCP 443 to the TURN domain. Optional STUN servers need their advertised UDP ports.
Web server inbound	TCP 443; TCP 80 only if using the supplied redirect or HTTP certificate validation. Never expose TCP 8000.
TURN server inbound	TCP 443 and UDP 49160-49259. These examples disable plaintext TCP/UDP client listeners and DTLS.
TURN server outbound	UDP to permitted public peer/relay IPs and ports, plus replies on client TCP/TLS connections. Restrict private/special destinations as described below.

TCP/TLS is the **browser-to-TURN** transport. Coturn still allocates UDP relay sockets and exchanges traffic with ICE peer endpoints; blocking the relay range breaks this setup. `no-tcp-relay` disables RFC 6062 TCP allocations, not the browser's TCP/TLS connection. Optional `turn:HOST:3478?transport=udp` requires enabling the UDP client listener and opening UDP 3478 in addition to the relay range. Coturn transport and port options.

Port 443 alone does not imply permission: a firewall may allow HTTPS while rejecting TURN TLS, unapproved domains or WebRTC. HTTP-only forward proxies, TLS inspection, packet loss, browser restrictions and captive portals can still prevent pairing. Use an IT-approved service/network when necessary; do not disable certificate verification or attempt to bypass policy.

Install the private Python service

Install the application under `/opt/lanflux` using an unprivileged `lanflux` service account. Keep the application and its virtual environment owned by the deploying administrator, readable/executable by the service user, and not writable by web clients. Create the environment and install dependencies using the project's setup instructions:

```
cd /opt/lanflux
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

Copy the public environment template to `/etc/lanflux/lanflux.env`. Replace all example domains and the secret. The same secret must appear in the private coturn configuration; it must not appear in browser code, public config responses, command-line arguments, logs or Git. Generate one privately, for example with `openssl rand -hex 32`, and place it directly into the protected configuration files. Suggested ownership: `/etc/lanflux` root:lanflux mode 0750, environment file root:lanflux mode 0640.

Install lanflux.service.example as `/etc/systemd/system/lanflux.service`. Check its paths and account, then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now lanflux
curl --fail http://127.0.0.1:8000/health
```

Keep one worker and one instance: rooms and AI state are in memory. A restart drops rooms and active signaling. Do not add load balancing or multiple workers without implementing shared room/signaling state. The unit uses the supported CLI, safe application logging and exact loopback proxy trust; no arbitrary forwarded-header trust is needed. Uvicorn proxy-header settings.

Configure HTTPS and WebSockets

Obtain a publicly trusted certificate for `share.example.com`, with automatic renewal. Certificate provisioning depends on your DNS/ACME provider; it is not automated here. Install the Nginx template in Nginx's `http` configuration context, replacing hostnames and certificate paths. Its `map` and rate-limit zones must be outside a `server` block. Keep default sites from unintentionally exposing the upstream or matching other hostnames.

The template requires HTTP Basic authentication over HTTPS. Create `/etc/nginx/lanflux.htpasswd` with the platform's `htpasswd` tool and separate strong credentials for approved users, or replace this with your organization's authenticated edge. Keep the password file root-owned and readable by the Nginx worker group, mode 0640. Never put passwords in invitation URLs. Test authentication and WebSocket upgrade in the actual target browsers before rollout; an SSO edge must preserve authenticated WSS access too. LANFLUX itself has no user accounts. Room codes and Origin validation do not substitute for this access control.

The exact AI routes `/api/aiml/sample`, `/api/aiml/reset` and `/api/aiml/export` use the application's live-peer Bearer token and Origin checks instead of Basic authentication, because one Authorization header cannot carry both schemes. They remain protected by backend authorization. The proxy forwards only Bearer credentials to Python and strips Basic passwords. Every other path, including all room APIs and WebSocket admission that issues TURN credentials, keeps edge authentication. Do not widen the exception to `/api/` or `/ws/`. Nginx variable authentication realms.

The proxy preserves Host and forwards Upgrade/Connection explicitly for WSS. It **replaces** client-supplied forwarded IP/protocol values with the actual connection values. Python accepts those values only from the exact local proxy IP. If there is another upstream proxy/CDN, configure its trusted real-IP chain deliberately; do not copy this example and trust every sender. Nginx WebSocket proxying.

```
sudo nginx -t
sudo systemctl reload nginx
curl --fail --user YOUR_APPROVED_USERNAME https://share.example.com/health
```

The last command prompts for the password. Do not use `curl -k` or browser certificate-warning exceptions. Public room URLs, QR codes, API requests and WSS must all use the exact configured HTTPS origin, with no subpath prefix. The proxy disables response caching and access logs for this service. Error logs can still contain sensitive request context; restrict access, retention and collection, and avoid verbose debug logging.

Configure coturn with expiring credentials

Use turnserver.conf.example, replacing the documentation IP, domain, certificate paths and shared secret. It targets coturn 4.6.x configuration syntax; use a currently patched distribution package and review its installed `turnserver -h`/manual. Newer releases may remove `no-tlsv1`, `no-tlsv1_1` or `no-cli` while disabling obsolete TLS versions/management listeners by default. Remove an obsolete directive only after checking the replacement behavior; verify that TLS 1.0/1.1 remain disabled. Do not enable the `tlsv1`/`tlsv1_1` compatibility flags on newer versions. Upstream coturn configuration reference.

Coturn uses `use-auth-secret` and `static-auth-secret`; the word "static" refers to the **server-held signing secret**, not a permanent public client password. LANFLUX issues timestamped usernames with Base64 HMAC-SHA1 credentials to admitted room peers in WebSocket messages. Public `/api/config` has availability flags and STUN configuration, without TURN credentials. The default credential lifetime is 600 seconds, configurable from 60-3600; keep web/TURN clocks synchronized. A replacement pairing receives fresh credentials. The expiration bounds use for new allocations; it is not a precise forced termination time for an already authenticated allocation. Coturn REST credential design.

Keep both static username/credential environment variables empty in shared-secret mode. Static credentials remain an option for controlled private deployments, but do not publish a reusable relay password. Do not use coturn `no-auth`, `server-relay`, `allow-loopback-peers`, plaintext downgrade URLs as a workaround, or wildcard peer exceptions. Room admission remains anonymous without the protected edge; expiry does not prevent abuse by someone who can repeatedly obtain admitted rooms. Apply edge access control, issuance/request limits, allocation quotas, bandwidth caps and provider spending alerts together.

The example denies loopback, private, link-local, carrier-grade NAT, multicast and other special peer targets, including cloud metadata address space. Add provider-specific internal/service ranges and enforce outbound firewall policy as a second layer. Do not override denied ranges with `allowed-peer-ip`: coturn gives explicit allows precedence. Private destination denial is intentional; this public relay is not an internal network access service. Coturn peer ACL and quota options.

Install a trusted full certificate chain for `turn.example.com`. Use a dedicated `/etc/lanflux-turn/tls` directory root:turnserver mode 0750 and a private key root:turnserver mode 0640; use the account name your distribution actually provides. Never make keys world-readable. Certificate renewal must securely update the readable copies and restart coturn according to the installed version's behavior. Ensure the certificate covers the TURN URL hostname. TLS 1.2 or newer is required.

Use the coturn systemd example if you are not using the distribution's maintained unit. Do not run both units on the same ports. This example runs as `turnserver` and grants only the low-port bind capability required for 443. Systemd creates private runtime/state directories for the PID file and coturn database. Protect the coturn configuration root:turnserver mode 0640. Logs go to the protected system journal; retain only what operations require. The sample quotas are small starting limits, not capacity guarantees: tune after measuring, and keep the relay port pool consistent with allocation limits.

Verify the deployed service

1. Confirm DNS resolves from both networks, both certificates validate, and `https://share.example.com/health` succeeds after edge authentication. Check that external connections to port 8000 fail.

2. Check TURN TLS independently: `openssl s_client -connect turn.example.com:443 -servername turn.example.com -verify_hostname turn.example.com -verify_return_error </dev/null`. This checks the TLS endpoint only; it does not prove TURN allocation, relay ports or file transfer.

3. On two authorized devices, open the HTTPS site, create/join a fresh room and transfer a small known file with receiver approval. Download it and compare its SHA-256 against the source. Inspect the observed route; `Unknown` is not proof of either direct or relay operation.

4. To test TURN, set Settings -> Relay only before the next connection on both browsers, leave/rejoin a fresh room and repeat the verified download. Require an observed **TURN relay** route. Restore the normal policy afterward. Test once across genuinely separate networks and once on the intended college Wi-Fi with permission.

5. Verify wrong/expired credentials cannot create allocations, private peer destinations are denied, quotas limit allocations, WSS survives idle heartbeats, certificate renewal works and a service restart cleans up rooms. Inspect logs without collecting room capabilities, credentials, file names or file bytes.

Do not claim a successful physical-phone, public-network or college-network test from local browser automation. The examples require deployment-specific validation; a local TLS coturn fixture is not evidence that your provider firewall, DNS, certificates or college policy permit the production route.

Troubleshooting

Symptom	Check
Website works; room WebSocket fails	Authenticated WSS, proxy Upgrade headers, exact Origin/Host, forwarded HTTPS scheme and trusted local proxy IP.
Both peers appear; ICE fails	TURN URL/hostname/certificate, clock skew, matching REST secret, actual credentials issued after room admission, client egress and server relay UDP rules.
TLS test works; relay transfer fails	TLS alone does not test allocations. Check quotas, denied peer ranges, UDP relay range, public/private address mapping and external firewall logs.
Works at home; fails on campus	Captive portal, client isolation, domain allowlist or TURN protocol filtering. Request an authorized network configuration from IT.
Intermittent failure after replacement/restart	Create a fresh pairing to obtain current credentials; confirm only one Python worker. Server restarts intentionally discard rooms.

08 / Reproducible network experiments

Maintained source: docs/NETWORK_EXPERIMENTS.md

This procedure collects observations from actual browser transfers. No benchmark results are bundled or generated without transfers. The automated Chromium loopback regressions are correctness checks, not Wi-Fi performance claims.

Prepare

1. Generate fixtures from the repository root: `python scripts/generate_test_files.py`. It creates `lanflux-1MiB.bin`, `lanflux-10MiB.bin`, `lanflux-50MiB.bin` and `sha256.json` in ignored `experiment-files/`. Python hashes bounded chunks independently; files are 1,048,576, 10,485,760 and 52,428,800 bytes. Existing files are preserved; use `--output ANOTHER_FOLDER` for fresh fixtures.

2. Start `python -m lanflux --lan --port 8000` on a trusted LAN. Open the computer's LAN URL on both devices. Configure HTTPS/WSS and suitable TURN if comparing internet networks.

3. Note the actual devices, browsers/versions, Wi-Fi band, distance, router, other traffic, VPN status, power mode and network condition. Do not infer link capacity from observed application throughput.

4. Pair the browsers. Check Network diagnostics for actual route and state. Record your conditions and run/direction information separately, or supply them to the developer export helpers below. These descriptions are self-reported.

Save observations with developer tools

Record values from Network diagnostics and Transfer details manually, or use the tested local export helpers through the browser developer console:

```
const {experimentsToCsv, buildExperimentReport, downloadText} = await
import('/static/experiments.js');
const conditions = {environmentLabel: 'Same Wi-Fi, laptop to phone', notes: '1 MiB, run 1'};
downloadText(experimentsToCsv(window.lanfluxClient.transfers, conditions), 'lanflux-run.csv',
'text/csv');
// Alternatively download a human-readable report:
// downloadText(buildExperimentReport(window.lanfluxClient.transfers, conditions), 'lanflux-
run.txt');
```

Replace the example descriptions with your actual conditions. These calls export existing terminal records, not fabricated measurements. Export before clearing finished records. Mobile browsers may require connected developer tools; manual recording remains an option.

Run a condition

Fixture	Repetitions	Maximum retained payload if uncleared
1 MiB	3	3 MiB
10 MiB	3	30 MiB
50 MiB	3	150 MiB - exceeds the 128 MiB receive limit

For each size, send the **same fixture three times** sequentially. Approve each file, keep both tabs foregrounded, wait for verified completion, then download and inspect its result. Export the report/CSV after **every run**, then clear finished files on both devices after saving/exporting. This prevents three 50 MiB

receives from exceeding the payload limit and keeps comparisons independent of retained memory pressure. Combine exported CSV rows locally without duplicating their header rows.

Use the receiver's records as the primary transfer-completion observations, since those count actual received payload. Optionally export sender observations too, clearly labeled; do not treat both directions' records for the same file as independent experimental runs. Sender duration includes acknowledgement latency. Keep failure/cancellation records with their reasons instead of quietly excluding them.

To measure setup repeatability, leave and recreate a room for **each run** and record the observed setup time. Reusing a connection is valid for steady-state throughput testing, but its setup figure is the original pairing observation, not a new setup measurement.

Repeat the procedure in the reverse direction. For comparison conditions, vary one thing at a time where practical: same Wi-Fi at different distances, a controlled busy network, or different supported networks. Record changes explicitly. Avoid comparing a foreground desktop run with a sleeping phone run as if only the Wi-Fi changed.

Optional TURN comparison

Configure a real relay using `.env.example`, manage relay credentials properly, and create a **new room** after selecting "Relay only" in Settings. Confirm Network diagnostics reports **TURN relay** from the selected candidate pair. If no relay route is observed, label the route unavailable or record the failure; do not assume a relay based on settings alone. TURN adds another network/service path and may be limited by that service, geography or quotas.

Remote connectivity can fail without a suitable relay. Automated loopback verification does not establish TURN or external-network performance.

Interpret the exports

Each terminal record includes file size, transferred payload bytes, setup milliseconds, transfer seconds, average/peak Mbps, result/reason, SHA-256 verification/digests and observed route. Reports omit room codes, TURN credentials and private ICE addresses. Names and user notes are local report data; share only what you intend.

- Average throughput (Mbps) = observed payload bytes x 8 / active transfer seconds / 1,000,000. For verified completed files, payload bytes equal file size. Failed runs may contain partial bytes.

- Transfer time starts at acceptance/start and ends at verified receiver completion or sender acknowledgement. It excludes the sender's prehash and human approval delay. Measure a separate wall-clock user workflow if you need end-to-end usability latency.

- Setup time starts when two peers are paired and ends at DataChannel open. It excludes waiting for the second person to join.

- Peak observed is the highest complete approximately one-second sampling interval. It is not instantaneous peak link bandwidth. Short runs may finish without a sample and report **Unavailable**; do not substitute zero or the average.

- RTT on the dashboard is explicit DataChannel ping/pong elapsed time, including queueing/scheduling. It is not ICMP or physical propagation delay.

- Integrity Verified means exact length and matching sender/received SHA-256 over actual bytes. Compare the sender digest against the generator's `sha256.json` when available. A match does not authenticate a real-world person or prove a file is safe.

- Sizes use binary MiB; network rates use decimal Mbps and MB/s. Browser send counters indicate bytes handed to the channel, while receive counters indicate bytes accepted; transport overhead is excluded.

For each size/condition/direction, report all three observed outcomes, the successful runs' median/range of throughput and duration, and how many failed. Three repetitions are a small student experiment, not a statistically strong general benchmark. Include unavailable metrics and explain timing differences rather

than inventing precision or results. Save screenshots with optional private ICE inspection hidden.

Manual computer-and-phone regression

- Open the LAN URL on both devices; create and join by room code, then repeat by QR/invitation.

- Send a small file computer -> phone and phone -> computer. Before approval no file bytes should transfer. Download and check **Integrity Verified**.

- Queue multiple files, include an empty file, reject one, and cancel from each side. A later file should still work.

- Leave/reload during a transfer. Unfinished items should fail cleanly; rejoin and select again. No resume is promised.

- Check diagnostics, export real report/CSV records, and hide private ICE details before screenshots.

- Save files before confirming "Clear finished files"; verify memory is released. Test the 50 MiB fixture only if the phone has sufficient available memory.

AI experiment: anomaly detection and throughput forecasting

Run AI experiments separately from short-file transfer benchmarks. A 1 MiB transfer can finish before a useful network sample; counting idle time around it as successful predictive evidence would be misleading. The AI sampler observes aggregate sent-plus-received payload throughput, so simultaneous bidirectional traffic changes the quantity being predicted. Use one transfer direction at a time for a simpler first experiment, then label bidirectional conditions explicitly.

Collection procedure

1. Pair devices, open Network diagnostics and select **Reset learning baseline** before starting a new experimental session. Reset deletes the current peer's observations and both models; export an earlier session first if it matters.

2. Record the device/browser versions, actual route, network condition and intended workload. Keep the tabs foregrounded. Do not expose optional private ICE details in screenshots.

3. Transfer actual representative files for approximately **70-90 seconds or longer** of active observations. Keep each file at or below 64 MiB and save/clear received files before the 128 MiB retained-payload cap. The time suggestion is not a guarantee of training.

4. Observe the two separate statuses. IsolationForest needs at least 30 observations, 25 server-measured seconds and three distinct rounded feature combinations. RandomForest needs 60 eligible labeled windows, meaningful speed variation and sufficient chronological training/validation examples after overlap purging.

5. Continue activity after training. Record measured and forecast Mbps, actual validation MAE, last-observed-speed baseline MAE, anomaly state and score. Export **real samples CSV** while still connected, before reset or departure.

6. Repeat under another authorized, clearly described condition. Change one factor at a time where practical. Keep both successful and failed runs and report unavailable values.

Fast transfers, pauses between files, missing RTT or irregular browser scheduling may leave the forecast in **Learning from network activity**. This is a valid result: report why the eligibility requirements were not met. Do not inject fake observations or use automated unit-test fixtures as training data for a demonstration. A controlled automated browser test verifies the pipeline, but its loopback workload is not a physical Wi-Fi performance measurement.

What the predictive evaluation measures

The regression target is the approximate duration-weighted payload throughput over the **three seconds after** each input anchor, conditional on continued transfer activity. Its features use only five already-observed samples: recent throughput, rolling mean/variation, trend, RTT/variation, activity and reliable route indicators. The target is built only when actual subsequent intervals arrive. The last interval is clipped proportionally at the horizon, so the label is an interval-rate approximation rather than exact sub-second ground truth.

Disconnected or invalid gaps break pending windows. Every input and future interval must be active; idle runs cannot create artificially low forecast errors. Valid forecasting samples have client intervals and server receipt gaps between 750 and 2000 ms, with a bounded timing discrepancy. Zero throughput during a genuinely active stall remains a real future observation.

The first 70% of eligible windows are training candidates and the final 30% validate. Earlier training examples whose labels overlap the first validation input interval are purged. The displayed errors are:

```
model MAE (Mbps) = mean(abs(actual future Mbps - model forecast Mbps))
baseline MAE (Mbps) = mean(abs(actual future Mbps - last observed Mbps))
```

Compare both errors on the same holdout. If the forest's MAE is larger, the baseline performed better in that session; state this plainly. Lower model MAE supports only a result on that particular holdout, not a general accuracy or speed-improvement claim. Overlapping windows are correlated, session sizes are small and route/workload changes can cause distribution shift. Independent longer sessions and explicit test conditions are needed for stronger conclusions.

A useful student table contains **condition, route, eligible labels, training rows, validation rows, model MAE, baseline MAE, and collection limitations**. Fill it only from actual observations. No benchmark numbers or pre-filled result table are supplied.

IsolationForest evaluation CLI

The existing CLI evaluates the anomaly detector, not the RandomForest regressor:

```
python -m aiml.evaluate /path/to/lanflux-network.csv --output network-predictions.csv
```

It uses the first 30 chronological observations as a baseline and requires at least ten later holdout observations. `--train-samples 60` instead needs at least 70 total rows. It validates bounded real-data CSV input, prints observed counts and score summaries, writes later predictions and refuses to overwrite an existing output. See AI monitor for input restrictions and Windows syntax.

Without independent ground-truth anomaly labels, accuracy, precision and recall are unavailable. Anomaly counts and scores describe the model's output; they do not prove a cyberattack, congestion cause or predicted network failure. Do not deliberately disrupt a college network to obtain an anomaly.

Keep the two export formats separate

Export	How to obtain it	Includes	Appropriate use
Terminal-transfer report / CSV	Local developer helpers above	File names, sizes, digests, observed transfer timing/rates, route, result and user notes	File-transfer experiment outcomes and independent digest comparison
AI observations CSV	**Export real samples CSV** in AI Network Monitor	Network metadata, server receipt times, anomaly outputs, forecast outputs and validation errors	Session-local AI analysis and the anomaly evaluation CLI

The transfer helper runs locally in the browser and does not upload its file-related fields. AI endpoints receive network metadata only and never receive filenames, digests or file content. Both formats omit room credentials and private ICE addresses. Review locally entered notes and file names before sharing transfer reports. The two CSV schemas are different: do not feed terminal-transfer CSV into `aiml.evaluate`.

There is no separate experiment-export form in the application dashboard. The removed form is not required for real measurements; use the documented developer helpers or manual records for transfer reports, and the working AI CSV button for telemetry.

Reproducibility checklist for a college report

- State the tested commit, operating systems, browser versions and whether tests used physical devices or loopback browsers.
- Separate connection setup, file hashing, human approval and payload-transfer durations.
- Report binary file sizes and decimal rate units consistently.
- Include three actual runs per file size, condition and direction; explain exclusions and record failures.
- Confirm route from observed selected-pair statistics; keep unknown values unavailable.
- Record phone power/sleep state, network conditions and any relay quotas that could affect results.
- Distinguish SHA-256 byte integrity from sender identity and file safety.
- For forecasting, retain the actual chronological holdout and baseline comparison; do not label MAE as classification accuracy.
- Clearly distinguish synthetic automated-test fixtures from measured session observations.
- Keep raw exports private unless reviewed; publish only authorized, non-sensitive evidence.

09 / Verification evidence and limitations

Maintained source: docs/VERIFICATION.md

Local checks below were executed on 21 October 2026. This record describes the current implementation extending published 0.3.0; it is not a new versioned release or a physical-network performance benchmark.

Automated results

Check	Local result
Python 3.12 / pytest	195 passed
Node 24 unit tests	51 passed
Existing Chromium suite	16 real scenarios plus their parent test passed
Actual local coturn suite	Four integration cases plus parent passed; zero skipped
Actual Nginx HTTPS/WSS suite	Three integration cases plus parent passed; zero skipped
Frontend references and syntax	125 unique IDs and 24 JavaScript files checked
Release file audit	Indexed/tracked paths and contents checked; secret-guard self-tests passed
AI CSV evaluation	Actual browser-exported observations; chronological baseline and holdout, no invented dataset

CI is configured to run Python 3.10 and 3.12, JavaScript/static checks, Chromium and actual browser CSV evaluation, plus a new isolated coturn/Nginx job. Remote results are tracked separately by commit in GitHub Actions; the table records executed local checks. The published 0.3.0 baseline passed 117 Python tests, 41 JavaScript tests and the existing browser suite. Current runs are available in GitHub Actions.

Reproduce

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
npm ci --ignore-scripts
npx playwright install chromium
npm run test:unit
LANFLUX_SCREENSHOT_DIR=output/playwright/screenshots npm run test:e2e
# Install coturn/Nginx locally first, or set the binary variables below:
npm run test:turn
npm run test:proxy
python scripts/check_frontend.py
python scripts/check_release.py --self-test
python scripts/check_release.py
python -m aiml.evaluate output/playwright/ai-observed.csv --output output/ai-holdout.csv
```

The browser fixture uses a temporary Python server on port 8765 and separate real browser contexts. Downloads are compared byte-for-byte. Generated evidence stays in ignored `output/playwright/`. By default selected real screenshots update `docs/screenshots`; set `LANFLUX_SCREENSHOT_DIR=output/playwright/screenshots` to keep committed images untouched, as CI does.

The optional suites use the installed `turnserver`/`nginx`, or explicit `LANFLUX_TURN_BINARY` / `LANFLUX_NGINX_BINARY` paths. They report a skip when dependencies are missing; set `LANFLUX_REQUIRE_NETWORK_SERVICES=1` to require execution, as CI does. Neither runtime package is bundled with LANFLUX. Test processes bind only loopback, use temporary credentials and certificates, and clean up afterward. Chromium pins only the generated test

certificate key; system trust and production certificate verification are unchanged. Coturn's loopback-peer exception exists only in this isolated fixture; production templates deny those destinations.

Actual infrastructure checks

- Native `all` policy keeps a direct path with a working relay configured.

- With direct candidates withheld only by the test harness, actual relayed file bytes complete and verify. This simulates candidate unavailability, not a physical firewall.

- An unavailable UDP TURN endpoint and a reachable TCP alternative produce an observed relay and verified download.

- Actual TURN TLS/TCP carries approved file bytes with HTTPS/WSS signaling. Listeners use ephemeral high ports locally; TCP 443 configuration is covered by URI tests, not an external port-443 connectivity claim.

- The shipped Nginx template passes its config check, rejects anonymous config/room/WebSocket access, and forwards exact AI Bearer routes to real backend authorization.

- Authenticated HTTPS/WSS QR/automatic joining, receiver approval, exact SHA-256/download bytes, actual AI observations, CSV export and reset succeed through Nginx. Tests respect edge rate limits rather than disabling them.

Regression coverage

- Secure room generation, expiration/cleanup, membership, Origin/Host validation, rate/resource limits and malformed signaling/state transitions.

- LAN-reachable QR invitations, consistent Copy Link/display URL, automatic link joining, invalid/full invitation recovery and platform-specific adapter detection.

- Actual bidirectional file transfer, approval, rejection, sequential queues, empty files, sender/receiver cancellation, channel/peer interruptions and retained-memory limits.

- SHA-256 known-answer vectors, vendored-source checksums, received-byte verification, corrupted/missing data rejection and cancellation during hashing/verification.

- Measured statistics, unit formatting, bounded charts, route classification and privacy-preserving exports.

- AI thresholds, frozen baseline, later predictions, active/stalled-transfer classification, strict metadata, authorization, pacing, bounded retention and chronological CSV evaluation.

- Real WebRTC-derived AI observations, training, prediction, download/export and reset in Chromium.

- RandomForest past-only features and subsequent measured labels, future-data leakage guards, gap/idle exclusion, bounded fitting, purged chronological validation, baseline MAE, advisor logic and safe unavailable/stale fallback.

- A real 8 MiB DataChannel file with test-only bounded sender pacing supplies varied active observations for forecast training in Chromium. The receiver verifies and downloads the original bytes; UI forecasts and validation errors are populated from actual measurements, then cleared when idle. No telemetry injection is used. This controlled loopback workload is not a Wi-Fi speed benchmark.

- Narrow mobile viewport overflow checks and cached-page cleanup/reinitialization. The latter dispatches persisted page lifecycle events to test the application's hook; it is not a claim of native BFCache behavior across engines.

Current execution details

- `.venv/bin/python -m pytest -q`: 195 passed, one Starlette/httpx dependency deprecation warning; 8.15 seconds.

- `npm run test:unit`: 51 passed, zero failed/skipped.

- `LANFLUX_SCREENSHOT_DIR=output/playwright/documentation npm run test:e2e`: 17 passed (16 child scenarios and parent), zero failed/skipped; approximately 146 seconds.

- `LANFLUX_REQUIRE_NETWORK_SERVICES=1 npm run test:turn`: 5 passed (four cases and parent), zero skipped, using the explicit local coturn binary and its extracted library path; approximately 29 seconds.

- `LANFLUX_REQUIRE_NETWORK_SERVICES=1 npm run test:proxy`: 4 passed (three cases and parent), zero skipped, using the explicit local Nginx binary; approximately 14 seconds.

- `python scripts/check_frontend.py`: 125 unique element IDs, navigation/control/assets/CSP references and 24 JavaScript syntax checks passed.

The first attempt to invoke the extracted coturn binary lacked its shared-library path and correctly failed the required-service gate. Supplying that fixture's local library directory allowed the complete actual relay suite above to pass. No application security setting was disabled. Normally installed packages do not need that fixture-specific path.

The real exported CSV is also evaluated with `aiml.evaluate`; no ground-truth accuracy is inferred from a count of anomalies. Generated CSVs, certificates and browser artifacts remain ignored. Report generation is an optional documentation operation and does not change runtime dependencies. PDF layout is checked by rendering pages as images and inspecting the resulting pages.

Screenshots and privacy

Homepage, desktop/mobile dashboard, connected peer, successful verified receive, network diagnostics and trained AI panel images were captured from the running application. Browser tests mask room capabilities and invitation addresses. Measurement fields and transfer results are not fabricated. No physical-network address inspection is shown publicly.

The documentation gallery was refreshed on 22 October 2026 using the same real-browser regression suite. It now includes focused desktop/mobile views, QR pairing with capability redaction, receiver approval, active transfer, settings and observed diagnostics during the paced transfer. The screenshot manifest records every image and reproduction instructions. Screenshot-only framing does not alter file state, measurements or predictions. The refresh passed all 17 Chromium checks (16 scenarios and parent) with zero skips, plus 51 JavaScript unit tests and the frontend static checks.

Remaining verification

This upgrade does not claim fresh physical computer<->phone testing, Safari/Firefox coverage, public HTTPS hosting, college/cross-network reachability, publicly trusted TURN certificates or production relay operation, labeled anomaly accuracy, or general Wi-Fi throughput benchmarks. Mobile viewport emulation is not a physical phone. Previous physical-device success should be followed by the manual computer-and-phone regression before relying on a new deployment.

The installed FastAPI/Starlette test client currently emits one dependency deprecation warning while all Python tests pass; it does not indicate a failed application test.

10 / PDF font copyright and permission notice

Maintained source: docs/FONT_NOTICES.md

The PDF report embeds subsets of Bitstream Vera supplied by ReportLab. The browser application uses system fonts and does not download or bundle these report fonts. The following notice accompanies the embedded PDF typefaces.

Bitstream Vera Fonts Copyright

The fonts have a generous copyright, allowing derivative works (as long as "Bitstream" or "Vera" are not in the names), and full redistribution (so long as they are not *sold* by themselves). They can be be bundled, redistributed and sold with any software.

The fonts are distributed under the following copyright:

Copyright =========

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Bitstream" or the word "Vera".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Bitstream Vera" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.

Copyright FAQ ============

1. I don't understand the resale restriction... What gives?

Bitstream is giving away these fonts, but wishes to ensure its competitors can't just drop the fonts as is into a font sale system and sell them as is. It seems fair that if Bitstream can't make money from the Bitstream Vera fonts, their competitors should not be able to do so either. You can sell the fonts as part of any software package, however.

2. I want to package these fonts separately for distribution and

sale as part of a larger software package or system. Can I do so?

Yes. A RPM or Debian package is a "larger software package" to begin with, and you aren't selling them independently by themselves. See 1. above.

3. Are derivative works allowed?

Yes!

4. Can I change or add to the font(s)?

Yes, but you must change the name(s) of the font(s).

5. Under what terms are derivative works allowed?

You must change the name(s) of the fonts. This is to ensure the quality of the fonts, both to protect Bitstream and Gnome. We want to ensure that if an application has opened a font specifically of these names, it gets what it expects (though of course, using fontconfig, substitutions could still could have occurred during font opening). You must include the Bitstream copyright. Additional copyrights can be added, as per copyright law. Happy Font Hacking!

6. If I have improvements for Bitstream Vera, is it possible they might get

adopted in future versions?

Yes. The contract between the Gnome Foundation and Bitstream has provisions for working with Bitstream to ensure quality additions to the Bitstream Vera font family. Please contact us if you have such additions. Note, that in general, we will want such additions for the entire family, not just a single font, and that you'll have to keep both Gnome and Jim Lyles, Vera's designer, happy! To make sense to add glyphs to the font, they must be stylistically in keeping with Vera's design. Vera cannot become a "ransom note" font. Jim Lyles will be providing a document describing the design elements used in Vera, as a guide and aid for people interested in contributing to Vera.

7. I want to sell a software package that uses these fonts: Can I do so?

Sure. Bundle the fonts with your software and sell your software with the fonts. That is the intent of the copyright.

8. If applications have built the names "Bitstream Vera" into them,

can I override this somehow to use fonts of my choosing?

This depends on exact details of the software. Most open source systems and software (e.g., Gnome, KDE, etc.) are now converting to use fontconfig (see www.fontconfig.org) to handle font configuration, selection and substitution; it has provisions for overriding font names and subsituting alternatives. An example is provided by the supplied local.conf file, which chooses the family Bitstream Vera for "sans", "serif" and "monospace". Other software (e.g., the XFree86 core server) has other mechanisms for font substitution.